LEARNING TO IGNORE WHAT MATTERS: JEPAS IN STOCHASTIC WORLDS

Anonymous authors
Paper under double-blind review

ABSTRACT

Joint-embedding predictive architectures (JEPAs) learn world models by predicting future representations rather than future pixels, and the objective alone decides what those representations keep. We show that in stochastic worlds this freedom works against the controller through two independent mechanisms. The first is representational: the squared next-embedding loss rewards whatever is predictable, so when the controlled state carries fresh unobservable innovations and a nuisance does not, the objective learns to ignore the state. In a matched three-seed study on a minimal red-cube world, the same spatial JEPA that localizes the cube to 0.12 pixels under per-frame background noise loses it entirely (21.3 pixels) once hidden wind is added, while persistent noise under the same wind keeps it at 0.67 pixels: the effect of nuisance persistence reverses with the dynamics. A pixel policy trained inside the failing model's imagination dies in every evaluation episode, sooner than a blind random walk, because the model's own termination head reads death on every predicted latent. The second mechanism is predictive: a squared loss yields the conditional mean, and two exact examples show that a correct mean can misrank actions and that a correct one-step mean need not compose into a multi-step mean. In a gate world whose safe passage is revealed late, the JEPA's frozen latent supports 0.97 survival when a policy is trained on real transitions through its own decoder, yet the same policy trained inside its imagination survives 0.02 of the paired episodes, against 0.97 inside a flow-matching model trained on the same data. In Craftax under matched data and training budgets,

Internal draft: result pending. tool-acquisition outcome, from the inventory campaign

. These results establish that avoiding representation collapse and accurately predicting latent states are insufficient for successful stochastic control: distributional dynamics repair the second mechanism only when the representation has survived the first.

1 INTRODUCTION

A world model is supposed to discard what does not matter. Joint-embedding predictive architectures (JEPAs) make this a learning problem: an encoder and a predictor are trained together so that the predictor forecasts the encoder's own next output, and nothing in the objective asks for pixels back (LeCun, 2022; Assran et al., 2023; Bardes et al., 2024; Maes et al., 2026). The promise for physical intelligence is a compact, reusable description of the world, free of texture and lighting. The difficulty is that the same objective decides two things at once: *what* to encode and *how* to predict it. We show that in stochastic worlds it can decide both against the controller, for two reasons that are independent of each other and fail in different places.

The first reason is representational. Squared next-embedding error is minimized by keeping predictable features and dropping unpredictable ones. A background fixed for an episode is perfectly predictable even if the object that the agent controls is ignored (Sobal et al., 2022). Less obviously, the controlled object itself becomes the unpredictable part as soon as its dynamics carry fresh innovations that no history reveals. The objective then has a reason to *learn to ignore what matters*, and it does so without collapsing: the latent stays non-degenerate, the Gaussian regularizer is satisfied and the prediction loss is low.

The second reason is predictive. A squared loss produces the conditional mean of the next latent. Two exact calculations show what a planner loses by working with that mean even when it is correct. An action whose outcomes straddle the goal has its mean exactly on the goal, so a planner scoring the mean prefers it to an action that reliably lands close. And an exact one-step mean, applied recursively, need not equal the multi-step mean whenever the dynamics are nonlinear. Neither failure is an optimization failure, and no better estimate of the same mean repairs it.

We make both mechanisms measurable in a deliberately small visual world: one red cube, four movement actions, a background that is either redrawn every frame or held for the episode, and optional stochastic wind or an approaching wall whose opening is revealed late. Position, goal and executed path are directly visible, so representation and planning can be separated without leaning on an aggregate game score. The main empirical findings are:

1. **Nuisance persistence reverses under stochastic dynamics.** With deterministic motion, per-frame background noise gives the more accurate position read-out (0.115 versus 0.764 pixels). With hidden wind the ordering flips and the per-frame model loses the cube altogether (21.33 versus 0.668 pixels). Architecture, latent dimension, data, updates and three model seeds are matched. A pixel policy trained inside the failing model's imagination dies in every evaluation episode, sooner than a blind random walk, because the model's own termination head reads death on every latent its predictor produces; the same policy inside a flow model trained on the same data survives 0.13 to 0.95 of the episodes (Section 4).
2. **A latent that holds the state can still fail imagination.** In the gate world the frozen JEPA latent, read through its own decoder, supports 0.97 survival when the policy is trained on real transitions; the same network, budget and evaluation keys inside the JEPA's imagination give 0.02, and inside a flow-matching model trained on the same data 0.97. The flow model is the distributional reference, and decoded-real controls qualify both interfaces before any imagination result is read (Section 5).
3. **The consequence survives outside the toy.** In Craftax, under matched data and training budgets and a shared pixel policy,

   Internal draft: result pending. state the inventory-campaign outcome

   (Section 6).

Section 2 gives the two mechanisms with their exact examples and the Gaussian nuisance construction that makes the first one precise. The conclusion is not that JEPAs cannot model stochastic worlds; it is that avoiding representation collapse and predicting latents accurately are insufficient for stochastic control, and that fixing the predictor helps only if the representation has kept the state in the first place.

2 Two ways a predictive objective can ignore what matters

The objective. An encoder f_θ maps an observation x_t to $z_t \in \mathbb{R}^D$; a predictor g_θ maps the recent latents and actions to a forecast of z_{t+1}. LeWorldModel (Maes et al., 2026), the recipe studied here, trains both jointly with

$$\mathcal{L} = \frac{1}{D}\,\widehat{\mathbb{E}}\,\|g_\theta(z_{\le t}, a_{\le t}) - z_{t+1}\|^2 + \lambda\,\mathrm{SIGReg}(\{z_t\}), \tag{1}$$

where SIGReg is a sketched test that the latent marginals are standard Gaussian, and there is no stop-gradient, teacher network, decoder or reward. Two choices are hidden in this one line: the target z_{t+1} is chosen by the encoder, and the loss is a squared error.

2.1 Mechanism 1: the encoder chooses predictability over state

For a fixed encoder and an unrestricted predictor, the minimum squared prediction risk is the expected conditional variance of the target,

$$\inf_g \mathbb{E}\|g(C, A) - Z'\|^2 = \mathbb{E}\,\mathrm{trCov}(Z' \mid C, A).$$

Joint training lets the encoder shrink this quantity by choosing *what* Z' is. Write the observation as $x_t = R(p_t, b)$, where p_t is the controlled state and b a nuisance fixed for the episode. If b is

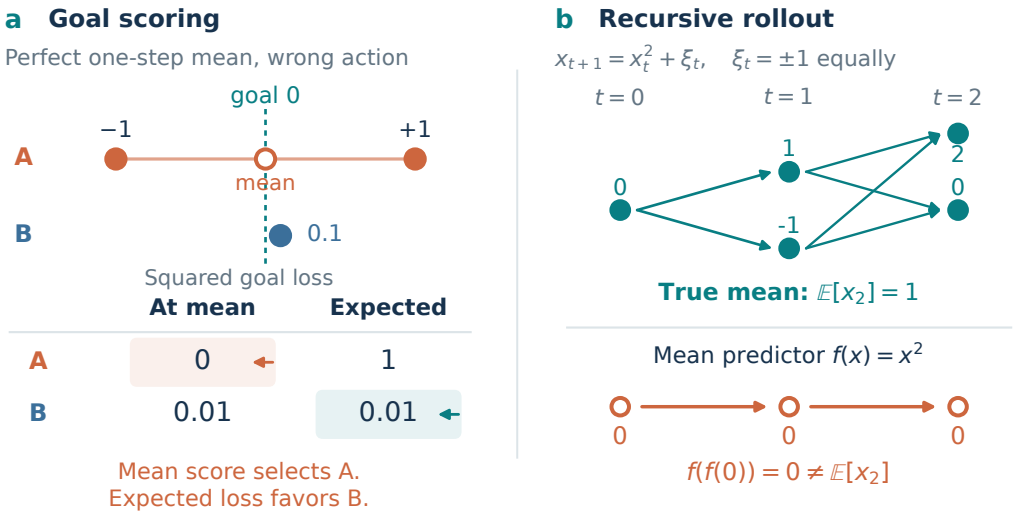


Figure 1: **A correct one-step mean can still give a wrong decision and a wrong rollout.** Left: the mean of action A sits on the goal although every outcome of A is farther from it than B's deterministic outcome. Right: for $X_{t+1} = X_t^2 + \xi_t$ with fair $\xi_t \in \{-1, +1\}$ and $X_0 = 0$, recursive conditional-mean prediction stays at 0 while the true two-step mean is 1. Both are exact calculations, not outputs of trained models.

recoverable and rich enough to be mapped to $\varphi(b) \sim \mathcal{N}(0, I_D)$, an encoder that returns only $\varphi(b)$ is fully non-collapsed, satisfies the Gaussian regularizer exactly, and lets the copy-last predictor reach zero loss. It is a global optimum of Equation 1 that carries no information about p_t (Proposition 1, extending Sobal et al., 2022; proofs in Appendix B).

The construction needs a constant nuisance. What carries over to stochastic worlds is the cost side: the innovations of the state are paid for in prediction loss, and an encoder that keeps the state at a fixed geometric scale must pay them (Corollary 1). Fresh unobservable noise in the dynamics therefore makes the controlled state the *expensive* part of the latent. Non-collapse says nothing about whether the encoder pays that price or quietly stops encoding the state. Section 4 measures whether it pays, and finds that the answer depends on the background in a way the zero-loss construction alone does not predict.

2.2 Mechanism 2: The squared loss chooses the mean over the distribution

Even with the state preserved, a squared loss learns $m(c, a) = \mathbb{E}[Z' \mid c, a]$. A goal planner needs expected costs. For a latent goal z_g,

$$\mathbb{E}\big[\|Z' - z_g\|^2 \mid c, a\big] = \|m(c, a) - z_g\|^2 + \mathrm{trCov}(Z' \mid c, a), \tag{2}$$

and an action-dependent second term can change the argmin. Figure 1 (left) is the smallest exact instance: the goal is 0, action A yields ± 1 with equal probability, action B yields 0.1 deterministically. The means are 0 and 0.1, so a planner scoring means picks A; the expected costs are 1 and 0.01, so B is right. No better estimate of the same mean repairs this; the omitted variance is exactly the missing quantity.

The mean does not compose. Let $X_{t+1} = X_t^2 + \xi_t$ with independent fair $\xi_t \in \{-1, +1\}$ and $X_0 = 0$. The exact conditional mean is $m(x) = x^2$, so recursive mean prediction gives $0, 0, 0, \ldots$ In the real process $X_1 \in \{-1, 1\}$ and $X_2 \in \{0, 2\}$, each with probability $1/2$, so $\mathbb{E}X_2 = 1$ (Figure 1, right). In general

$$\mathbb{E}[X_{t+1}] = (\mathbb{E}X_t)^2 + \mathrm{Var}(X_t), \tag{3}$$

and recursive substitution keeps only the first term. A JEPA rolled forward on its own mean predictions is doing exactly this substitution.

What repairs mechanism 2, and what does not. A model that predicts expected costs or values directly can keep the needed expectations without samples (Wan et al., 2019; Grimm et al., 2021); for the quadratic cost in Equation 2 a mean plus one scalar variance per action suffices. A distributional world model, such as a flow-matching predictor that samples Z', is the more general repair: it keeps distinct outcomes and averages their costs. Neither repair touches mechanism 1. If the encoder has already dropped the state, a perfect distribution over the remaining features is a perfect distribution over nothing useful. This ordering is the paper's central prediction, and Sections 5 and 6 test it with a flow model trained on the same data and budget as the JEPA.

3 WORLDS, MODELS AND THE COMPARISON CONTRACT

The red-cube worlds. A red 6×6 cube moves in a 63×63 RGB frame. Four actions move it three pixels left, up, right or down. Touching the border ends the episode with reward -1; all other rewards are zero and episodes are capped at 128 steps. The background is gray noise, either redrawn every frame (*per-frame*) or drawn once per episode (*persistent*). Optional *wind* is a hidden two-dimensional random walk: each coordinate moves by $-1, 0, +1$ with probabilities $0.1, 0.8, 0.1$, is clipped to $[-2, 2]$ and is added to the action displacement. Wind is never observed, so even a perfect history leaves fresh uncertainty about the next position. Crossing the four gray conditions with deterministic or wind dynamics gives the matched study of Section 4.

The *gate* world adds a wall that approaches the cube with a single opening at the top or the bottom, sampled at spawn and revealed only after a fixed delay. Surviving requires retreating, observing the opening and then committing. The solid part of the wall is common to both futures, so a model that keeps state but averages over the two openings should still show the wall; a model that loses the wall has lost state, not just resolved uncertainty in the wrong way.

The JEPA. The encoder produces $z_t \in \mathbb{R}^{512}$ arranged as a $4 \times 4 \times 32$ grid; a factored spatial-temporal predictor conditions on eight past latents and actions. Encoder and predictor are trained jointly with Equation 1, $\lambda = 0.09$, and nothing else. Each model sees 500,000 uniformly random transitions for 8,500 updates, about ten minutes on one A100. Three independent model seeds are fitted in the matched nuisance study. Pixel decoders, position read-outs and reward or termination heads are fitted *afterwards* on the frozen latent; their labels and compute are additional and are reported as such.

The distributional reference. The flow recipe uses a reconstruction-trained tokenizer with the same $4 \times 4 \times 32$ latent shape and a causal flow-matching predictor that *samples* the next latent (Lipman et al., 2023; Hafner et al., 2025). It is trained on the same transitions under the same wall-clock budget, with reward and termination heads fitted jointly. The two recipes differ in more than deterministic versus sampled prediction: the flow tokenizer is supervised by reconstruction, and its task heads send gradients into its dynamics trunk, whereas the JEPA is reward-free with post-hoc heads. Every JEPA-versus-flow gap in this paper is therefore a *recipe* gap. We narrow it where we can, by refitting the JEPA heads on all 500,000 training labels, and we say so where we cannot.

Behavioral comparison contract. Policies are compared through one interface only. Both families expose decoded RGB frames from their own frozen decoders to the same pixel CNN and GRU policy, trained by the same PPO implementation with the same optimizer, updates, batch and horizon, and evaluated with the final checkpoint on 256 stored episodes whose reset, texture and action keys are shared across families and seeds. Episodes are admitted by surviving a seven-step random prefix, never by policy outcome. Before any imagination result is read, the same policy is trained on *real* transitions observed through each model's decoder; a family whose decoded-real control fails has an interface problem, not an imagination problem. Rewards and terminations inside imagination come from each model's own heads; no simulator quantity enters training.

Craftax. Craftax-Classic (Matthews et al., 2024) is the consequence outside the toy: 17 actions, procedurally generated maps, sparse rewards for 22 achievements. Both world models are trained on the same 500,000-transition archive under the same ten-minute budget, and the same pixel policy is trained inside each for the same number of imagined steps before native evaluation (Section 6). Complete settings are in Appendix D.

Table 1: **The effect of nuisance persistence reverses under stochastic dynamics.** Cube position L1 error in pixels, mean ± sample standard deviation over three model fits. "Real" reads encodings of held-out observations; the right column applies the same read-out eight predicted steps ahead. Architecture, latent size, data and updates are fixed across cells.

Gray background / dynamics	Real encoding	Eight-step prediction
Per-frame / deterministic	0.115 ± 0.012	4.84 ± 1.13
Per-frame / wind	21.330 ± 0.057	20.39 ± 0.92
Per-episode / deterministic	0.764 ± 0.068	1.83 ± 0.22
Per-episode / wind	0.668 ± 0.049	10.95 ± 0.53

4 Mechanism 1 in the cube worlds: the state is kept only while it is predictable

Design. Four gray-noise conditions cross background persistence (per-frame versus persistent) with dynamics (deterministic versus hidden wind). Each condition receives three independently initialized grid JEPAs with identical architecture, 512-dimensional latent, 500,000 transitions and 8,500 updates. After training, a nonlinear position read-out is fitted on frozen encodings of training frames and scored on held-out frames as L1 error in pixels; the same read-out is then applied to eight-step predicted latents (Appendix C). Nothing about the read-out depends on the condition, so a difference between cells is a difference in what the encoder exposes.

The reversal. With deterministic motion the objective keeps the cube in every condition, and per-frame noise is the *easier* background: 0.115 ± 0.012 pixels against 0.764 ± 0.068 for persistent noise (Table 1, Figure 5). Adding wind flips the ordering and breaks one cell outright. Under persistent noise the read-out still localizes the cube to 0.668 ± 0.049 pixels; under per-frame noise it fails at 21.33 ± 0.06 pixels, an error larger than the cube is wide in every seed, with almost no seed-to-seed spread. The failure is already present in the encodings of real frames, before any prediction is made: decoded real encodings from a failing model show a gray field with at most a faint trace of the cube, and its predicted future is not a scene at all, while the flow model trained on the same data keeps the cube in both (Figure 2). The eight-step prediction column tells the same story one step later: where the state was kept, prediction error grows with the horizon as the wind's uncertainty accumulates; where it was dropped, prediction has nothing left to lose.

Why this is mechanism 1. Under deterministic motion the next position is a function of the history and the action, so keeping it makes the target predictable and lowers the loss. Wind removes that incentive: the position's innovations cost prediction loss no matter how the encoder represents them (Corollary 1). The failing cell is not the zero-loss nuisance optimum of Proposition 1: its prediction loss stays high, about 0.733 per coordinate, SIGReg is satisfied and the latent is far from degenerate. The objective did not find a cheap substitute for the state; it simply stopped paying for the state once paying no longer bought predictability. Why persistent noise protects the cube under the same wind is not something the theory decides. The matched design establishes only that it does, in all three seeds, and that the protection is an interaction between nuisance and dynamics rather than a property of either.

The state is encoded through the target. A second intervention isolates where the incentive acts. In the per-frame conditions we refit three seeds per world with the prediction target detached from the encoder's gradient, changing nothing else (the predictor-input and SIGReg gradients remain). With deterministic motion the detached models lose most of the position information that the joint models keep; with wind, both variants fail identically (Table 2). The encoder keeps the cube only when the joint gradient can make the cube's future predictable, and the wind failure is not a side effect of that gradient path. Detaching the target, the natural first repair, does nothing here.

Behavioral consequence. A policy that cannot see the cube cannot steer it, and a policy trained inside a model that cannot see the cube fares worse than one that does not look. Table 3 trains one stock pixel policy by PPO inside each model's imagination, fed by the model's own decoder, for 50

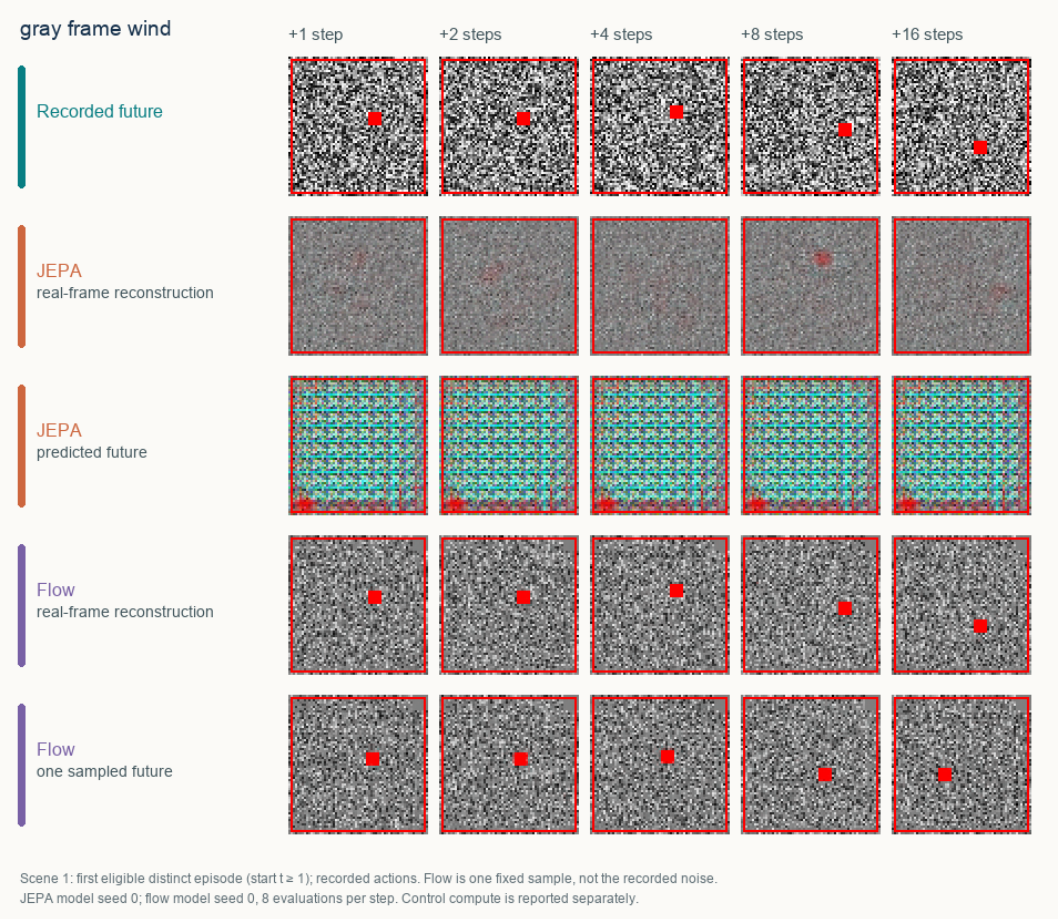


Figure 2: **Per-frame noise plus wind: the JEPA no longer encodes the cube.** Recorded future under the recorded actions; the JEPA's decoded encodings of those real frames; the JEPA's decoded rollout; the flow model's decoded encodings and one sampled future. First eligible episode of the held-out set, chosen without looking at any model output; JEPA model seed 0, flow seed 0. The decoders are supervised reconstruction probes fitted after training, so the middle rows show what is recoverable from the frozen latent, not what the model was trained to produce.

Table 2: **Detaching the target does not rescue the wind cell and costs the deterministic one.** Position read-out error (mean absolute pixel error per coordinate, read-out fitted on 50,000 frames; a different read-out protocol from Table 1), mean ± sample standard deviation over three model seeds; each seed pair shares initialization and data order.

Per-frame gray noise	Joint target	Detached target
Deterministic	0.37 ± 0.05	10.93 ± 4.90
Wind	14.51 ± 0.12	14.51 ± 0.07

updates and three policy seeds, and evaluates it on 256 paired episodes (protocol in Section 3). In the per-frame still world both imaginations produce policies near the ceiling set by blind alternation, so survival there certifies nothing. In the per-frame wind world every JEPA policy dies in every episode after 12.5 ± 3.1 steps of control, fewer than a uniformly random walk (17.4) or blind alternation (18.2) last; the flow policies trained on the same data survive 0.945, 0.129 and 0.652 of the episodes, better than their paired JEPA seed each time, with a seed spread that the 50-update budget does not close. Figure 3 shows what the JEPA policy was trained on: its termination head, which tracks the recorded rate on encoder latents, reads 0.91 on every latent the predictor produces, so an imagined

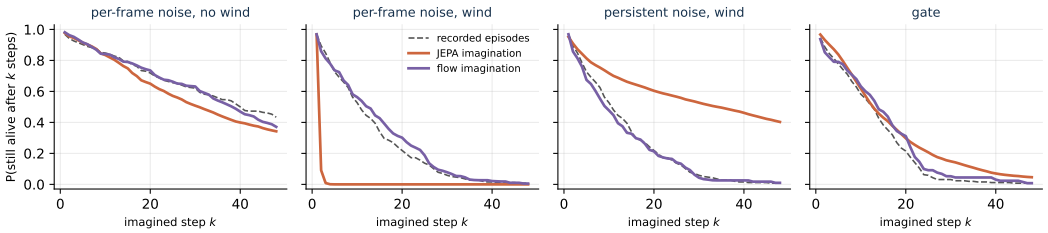


Figure 3: **What each model's imagination tells a policy about dying.** Each frozen model is rolled forward from 256 real contexts under the recorded (uniformly random) actions and its own termination head is read on its own predicted latents; the curve is the implied probability that the imagined episode is still alive after k steps, against the recorded survival of the same episodes (dashed). In the per-frame-noise wind world the JEPA head reads a 0.91 termination probability on every predicted latent from the first imagined step, while on encoder latents of the real frames it tracks the recorded rate (0.03–0.06); the model has no state left for the head to read, so imagination ends in two steps whatever the action. Under persistent noise the JEPA keeps the cube and its deterministic rollout is optimistic instead: the hidden wind's excursions are averaged away (Appendix J).

Table 3: **One pixel policy, four worlds, two imaginations.** Survival and control length (of 121 possible steps) of the stock Impala-CNN and GRU policy over 256 paired evaluation episodes, mean $\pm$ sample standard deviation over three policy seeds. "Imagined r" is the reward per step that the model paid the policy at its last training update; the true reward is never positive. "Real" rows train on real transitions seen through each model's decoder. Two state-blind rules run through the same evaluator give the scale (Appendix J).

World	Training / model	Survival	Length	Walls	Imagined r	Seeds
per-frame noise, no wind	imagined, JEPA	0.85 ± 0.18	109.3 ± 16.5	–	0.15 ± 0.08	3
per-frame noise, no wind	imagined, flow	0.97 ± 0.02	119.8 ± 0.8	–	-0.00 ± 0.00	3
per-frame noise, no wind	*uniform random*	0.24	74.0	–	–	–
per-frame noise, no wind	*left–right alternation*	1.00	121.0	–	–	–
per-frame noise, wind	imagined, JEPA	0.00 ± 0.00	12.5 ± 3.1	–	-0.75 ± 0.00	3
per-frame noise, wind	imagined, flow	0.58 ± 0.41	91.4 ± 33.5	–	-0.00 ± 0.00	3
per-frame noise, wind	*uniform random*	0.00	17.4	–	–	–
per-frame noise, wind	*left–right alternation*	0.00	18.2	–	–	–
persistent noise, wind	imagined, JEPA	0.02 ± 0.01	26.8 ± 8.7	–	-0.03 ± 0.00	3
persistent noise, wind	imagined, flow	0.41 ± 0.42	79.0 ± 33.1	–	-0.02 ± 0.01	3
persistent noise, wind	*uniform random*	0.00	17.4	–	–	–
persistent noise, wind	*left–right alternation*	0.00	18.2	–	–	–
gate	real, decoded JEPA	0.97 ± 0.02	119.7 ± 0.7	3.94 ± 0.03	–	3
gate	real, decoded flow	0.81 ± 0.25	113.6 ± 8.5	3.72 ± 0.31	–	3
gate	imagined, JEPA	0.02 ± 0.01	42.1 ± 5.3	0.55 ± 0.28	-0.00 ± 0.00	3
gate	imagined, flow	0.97 ± 0.02	119.1 ± 1.3	3.92 ± 0.06	-0.00 ± 0.00	3
gate	*uniform random*	0.00	24.0	0.07	–	–
gate	*left–right alternation*	0.00	24.6	0.05	–	–

episode ends within two steps whatever the policy does and the gradient carries no information about actions. The trained policy is arbitrary, and an arbitrary but consistent action bias reaches the boundary faster than chance. Under persistent noise the encoder keeps the cube and the failure changes character: JEPA policies now outlast the blind rules (26.8 ± 8.7 steps) but still survive only 0.02 ± 0.01 of episodes, against 0.891, 0.207 and 0.137 for the flow, and the deterministic rollout that trained them is optimistic rather than dead, keeping 0.40 of imagined episodes alive at 48 steps where 0.008 of recorded ones are. That is the averaging of the hidden wind, the subject of the next section. One more column deserves notice: in the still world the JEPA's imagined reward climbs to $+0.15$ per step although the true reward is never positive. The policy is farming the refitted reward head while it survives; the same signature returns, much larger, in Craftax.

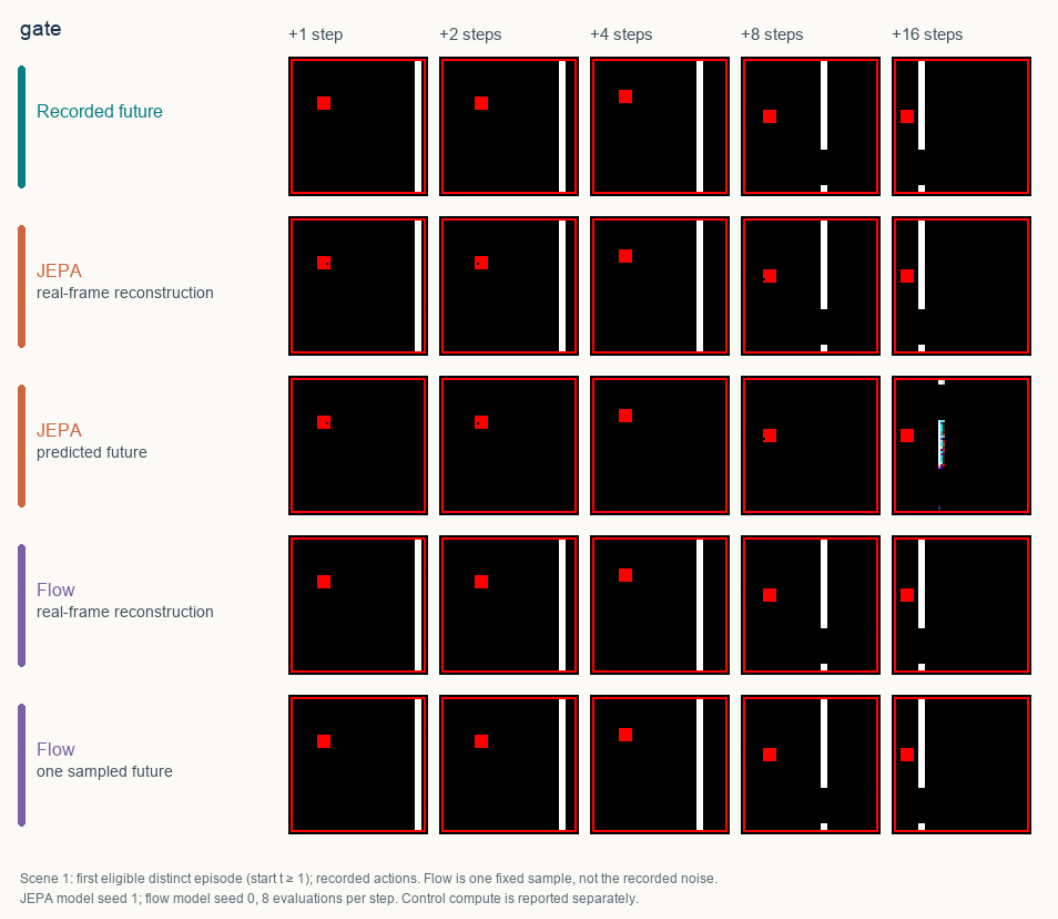


Figure 4: **The wall survives encoding and dies in prediction.** From one pre-reveal context and the recorded actions: the simulator; the JEPA's decoded encodings of the real future; the JEPA's decoded rollout; the flow model's decoded encodings and one sampled future. JEPA model seed 1, the encoder used in the behavioral comparison below; flow seed 0; first eligible held-out episode, chosen without looking at model outputs. A decoding failure on predicted latents mixes prediction error with off-distribution read-out; the behavioral comparison is what turns this picture into a claim.

5 Mechanism 2 in the gate world: a kept state that imagination cannot use

The gate world is built so that mechanism 1 is off the table. The background is black, the wall is large and slow, and the only uncertainty is which opening will be revealed. A model that keeps the state must show the approaching wall; a model that averages over the two openings should still show the shared solid part. Figure 4 shows what the grid JEPA does with a pre-reveal context: decoded encodings of the *real* future keep the wall and its opening at every step, and the predictor's own rollout deletes the wall from the first predicted step, in all three encoder seeds (Appendix L). The flow model's sampled rollout keeps the wall and commits to one opening. This is the visible signature of the second mechanism: the state was there, and the mean-trained predictor did not carry it forward.

Why a shared pixel policy. Earlier attempts to compare the two models through latent-input policies were uninformative in a specific way: a latent-MLP policy trained by PPO fails the gate even on the *exact* simulator with true rewards (0 of 45 policies above 50% survival; Appendix M), so its failure inside imagination says nothing about the model. The comparison in this section therefore uses one pixel CNN and GRU policy for both families, fed by each model's own decoder, with the

same PPO, 150 updates of 512 rollouts of 48 steps, three policy seeds, final checkpoints, and 256 evaluation episodes on shared keys (Section 3). The JEPA encoder is model seed 1, the seed on which decoded-latent control had succeeded during development; the flow model is one dynamics checkpoint. Both facts limit the claim to these fitted models. The JEPA's reward and termination heads are refitted on all 500,000 training labels so that head supervision is count-matched to the flow recipe.

Interface qualification. Before imagination is read, the same policy is trained on real transitions seen through each decoder (Table 3, "real" rows). Through the JEPA's decoder all three policy seeds solve the gate: survival 0.977, 0.980 and 0.945, passing 3.9 of a possible four walls per episode. The JEPA latent, read by a supervised decoder, therefore carries everything the task needs, and the policy and optimizer can learn the wait-and-commit strategy from it. Through the flow decoder two seeds solve it (0.957, 0.953) and one does not (0.516), so the flow interface qualifies but is less stable; an imagination result for flow below its best decoded-real seed is not automatically a model failure.

Imagination. The same policy trained inside the JEPA's imagination, with the same budget and evaluated on the same 256 episodes, survives 0.027, 0.008 and 0.027 of them and passes 0.55 ± 0.28 walls per episode (Table 3). It is not a random policy: it controls for 42 steps where the blind rules last 24, and it passes the first wall eight times as often as chance. It is a policy that has learned to avoid the border in a world without walls. Inside the JEPA's rollouts the fraction of imagined positions still alive rose from 0.34 to at most 0.45 over training while the policy's entropy fell to 0.03–0.14: PPO converged, confidently, on a strategy that the model rewarded and the simulator does not. Inside the flow model's imagination the same policy survives 0.988, 0.941 and 0.977 of the paired episodes and passes 3.92 ± 0.06 walls, matching its own decoded-real controls. Two facts bound the reading. The JEPA latent fed to the policy in imagination is the latent that supported 0.97 survival from real transitions, so the deficit is in the predictor's rollout, not the encoder or the interface; and the flow recipe differs from the JEPA in supervision stage as well as in sampling, so the gap is a recipe gap (Section 3). What the comparison establishes is the ordering the paper is about: the state was kept, and a mean-trained predictor applied to its own outputs did not carry it forward.

What the historical record adds. Across the earlier, unmatched protocols, none of 27 policies trained inside the JEPA's imagination discovered the gate strategy, including nine that used a sampling head fitted on the predictor's residual and nine that averaged eight of its samples, while policies trained on real transitions through the same frozen encoders did; the flow reference's historical success was a single policy of six (Appendix L). Both facts are why this section rests on the matched comparison rather than on those tables.

6 Craftax under matched data and budget

Craftax-Classic asks the same question at the scale of a game. Both world models are trained on the same 500,000-transition archive of uniformly random play under the same ten-minute budget, with the same $4 \times 4 \times 32$ latent shape. The same pixel CNN and GRU policy is then trained by PPO inside each model for five million imagined steps, from the same bank of real episode-start contexts, with rewards and terminations from each model's own heads and no behavior cloning, action masking or reward shaping. Each policy is evaluated in the real game on 256 episodes from a fixed seed. The primary endpoint is acquisition of a wooden pickaxe, a goal that needs a multi-step plan (collect wood, place a table, craft) and cannot be reached by staying alive or wandering; all 22 achievements, return and episode length are retained (Appendix I).

Internal draft: result pending. Insert the paired inventory-campaign outcomes: wood-pickaxe rate per policy seed for JEPA and flow, achievement count, return and length. Flow seed 0 so far: 242/256 pickaxes, 6.04 achievements per episode. Then read the result against the two mechanisms: does the JEPA's imagination support the crafting sequence at all, and where does it fall short?

The same 512-dimensional JEPA supports short-horizon goal-directed movement in Craftax when the goal is an image a few actions away (open-loop latent planning reaches 93% of nearby targets

against 17% for random actions; Appendix I). Local state is there. What the inventory task measures is whether imagination from that state can carry a plan.

7 Related work

Joint-embedding world models. Joint-embedding prediction avoids reconstructing observable detail (Assran et al., 2023; Bardes et al., 2024). LeWorldModel jointly learns encoder and action-conditioned predictor with a squared loss and a Gaussian regularizer (Maes et al., 2026; Balestriero & LeCun, 2025) and is the recipe we study. DINO-WM and V-JEPA 2 predict on frozen pretrained features (Zhou et al., 2024; Assran et al., 2025); PLDM learns the representation jointly with additional regularizers (Sobal et al., 2025). Sobal et al. (2022) study action-conditioned moving objects with fixed or changing distractors and identify the predictable-nuisance failure that our Proposition 1 specializes to the Gaussian-regularized objective. Our matched study adds fresh, unobservable innovations in the controlled state and finds that the effect of nuisance persistence reverses under them; this is a boundary in a different recipe, not a contradiction of their findings. Gulati & Nemenman (2026) examine slow-noise bias in contrastive learning.

Prediction versus planning geometry. LeWorldModel itself reports accurate probes alongside weaker planning. Temporal-Distance JEPA and RC-aux add temporal or reachability structure, and physically grounded variants add task-state supervision (Bai & Xiong, 2026; Li et al., 2026; Yan et al., 2026; Liu et al., 2026); Terver et al. (2026) study planner and architecture choices and note multimodal conditional means. Our exact examples isolate the decision consequences of scoring at a mean and of composing mean transitions, independent of any architecture.

Generative and distributional imagination. Generative world models have supported control in imagination (Hafner et al., 2020; Alonso et al., 2024; Hafner et al., 2025) and also exhibit perception, action-conditioning and rollout failures (Hansen & Wang, 2026). FlowWM learns stochastic feature-space futures without actions and reports no control (Porcher et al., 2026). Our flow reference is action-conditioned, trained on the JEPA's data and budget, and compared through a shared policy so that behavior, not video quality, is the endpoint.

Objective mismatch and value equivalence. That a model good for prediction need not be good for control is longstanding (Lambert et al., 2020). Value-aware model learning, expectation models and (proper) value equivalence fit the quantities a decision class needs (Farahmand et al., 2017; Wan et al., 2019; Grimm et al., 2020; 2021); TD-JEPA predicts policy-conditioned successor features (Bagatella et al., 2026). These are the reason our mean-state examples are not impossibility results for deterministic world models: a model can keep expectations without keeping samples, provided the representation still carries the state those expectations depend on.

8 Discussion

Two mechanisms, two places to look. The representation mechanism shows up before any rollout: a position read-out on *real* encodings fails, decoded real frames lose the cube, and no change to the predictor or planner can bring it back. The planning mechanism shows up only after the state has been kept: the gate latent supports control from real transitions but its imagination does not. A practitioner who sees a failed imagination policy should first check which of the two they are looking at, because the remedies are disjoint. Detaching the target, adding decoders, or changing what the encoder is asked to keep addresses the first; distributional or value-aware prediction addresses the second.

What the flow reference does and does not show. The flow model differs from the JEPA in supervision stage, not only in sampling versus point prediction, so its advantage where it has one is a recipe-level result. The ordering claim survives this caveat: a sampled predictor cannot restore a state that the encoder never kept, and where the JEPA representation held the state, the remaining gap is the kind that sampling addresses. We do not claim that generative modeling is necessary; for a quadratic goal cost a mean and a variance suffice, and value-equivalent models can carry the needed expectations directly.

Honest boundaries. The empirical per-frame-noise-plus-wind failure has high prediction loss and is not an instance of the zero-loss nuisance optimum of Proposition 1; the proposition explains why non-collapse cannot protect the state, not which finite-budget model drops it. The gate JEPA encoder used for the behavioral comparison is model seed 1, chosen because it was the only seed on which decoded controls had succeeded in development, and every flow gate policy shares one dynamics checkpoint. Decoded-latent results are statements about information recoverable by a supervised decoder, not about the unsupervised model alone. The worlds are small by design; Craftax shows the consequence at the scale of a real game but under a fixed ten-minute budget.

Outlook. General world models need a principle for choosing what to keep, and predictability is not it. The results here argue for objectives that price the controlled state's innovations as information to model rather than as loss to avoid, and for evaluating a learned world model at three separate points: what its representation retains, what its predictions preserve, and what a policy trained inside it can do.

Reproducibility statement

All environments, models, policies and evaluators are released with the code. Every reported policy number is a final checkpoint evaluated on stored keys; the budgets are enforced by the training scripts, which refuse other values. Model, data and source hashes are written next to every run. `AUDIT.md` in the repository lists, with file and line references, where each safeguard lives and which historical numbers it does not cover.

Use of large language models

Language-model coding agents wrote most of the experiment harness and earlier drafts of this text under the authors' direction; the authors specified the protocols, audited the code and results, and are responsible for every claim.

References

Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

Mahmoud Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning, 2025.

Marco Bagatella, Matteo Pirotta, Ahmed Touati, Alessandro Lazaric, and Andrea Tirinzoni. TD-JEPA: Latent-predictive representations for zero-shot reinforcement learning. In *International Conference on Learning Representations*, 2026. URL `https://arxiv.org/abs/2510.00739`.

Jiaxin Bai and Jiaxuan Xiong. Temporal-distance JEPA: Plan-aware representation learning for latent world model predictive control, 2026. URL `https://arxiv.org/abs/2607.25337`.

Randall Balestriero and Yann LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics, 2025.

Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video, 2024.

Boyuan Chen, Diego Martí Monsó, Yilun Du, Max Simchowitz, Russ Tedrake, and Vincent Sitzmann. Diffusion forcing: Next-token prediction meets full-sequence diffusion. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *International Conference on Machine Learning (ICML)*, 2018.

Amir-massoud Farahmand, Andre Barreto, and Daniel Nikovski. Value-aware loss function for model-based reinforcement learning. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 1486–1494, 2017.

Jesse Farebrother, Jordi Orbay, Quan Vuong, Adrien Ali Taiga, Yevgen Chebotar, Ted Xiao, Alexander Irpan, Sergey Levine, Pablo Samuel Castro, Aleksandra Faust, Aviral Kumar, and Rishabh Agarwal. Stop regressing: Training value functions via classification for scalable deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2024.

Christopher Grimm, André Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Christopher Grimm, André Barreto, Gregory Farquhar, David Silver, and Satinder Singh. Proper value equivalence. In *Advances in Neural Information Processing Systems*, volume 34, pp. 7773–7786, 2021. URL https://proceedings.neurips.cc/paper/2021/file/400e5e6a7ce0c754f281525fae75a873-Paper.pdf.

Paarth Gulati and Ilya Nemenman. Contrast encodes inductive bias: Separating slow noise from dynamics in predictive representation learning, 2026. URL https://arxiv.org/abs/2606.07770.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.

Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models, 2025.

Nicklas Hansen and Xiaolong Wang. Hallucination in world models is predictable and preventable, 2026. URL https://arxiv.org/abs/2606.27326.

Nathan Lambert, Brandon Amos, Omry Yadan, and Roberto Calandra. Objective mismatch in model-based reinforcement learning. In *Conference on Learning for Dynamics and Control (L4DC)*, 2020.

Yann LeCun. A path towards autonomous machine intelligence, 2022. Version 0.9.2, OpenReview.

Wenyuan Li, Guang Li, Keisuke Maeda, Takahiro Ogawa, and Miki Haseyama. Predictive but not plannable: RC-aux for latent world models, 2026. URL https://arxiv.org/abs/2605.07278.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

Muyuan Liu, Yue Huang, Zheng Liang, and Xiang Gao. Toward physically grounded JEPA world models for goal-conditioned robotic planning, 2026. URL https://arxiv.org/abs/2609.03565. IROS 2026 Workshop on Physical World Models for Scaling Embodied AI.

Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels, 2026.

Michael Matthews, Michael Beukman, Benjamin Ellis, Mikayel Samvelyan, Matthew Jackson, Samuel Coward, and Jakob Foerster. Craftax: A lightning-fast benchmark for open-ended reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2024.

François Porcher, Nicolas Carion, Karteek Alahari, and Shizhe Chen. Flow matching in feature space for stochastic world modeling, 2026. URL https://arxiv.org/pdf/2606.29059v2.

Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features, 2022. URL https://arxiv.org/abs/2211.10831.

Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models, 2025.

Basile Terver, Tsung-Yen Yang, Jean Ponce, Adrien Bardes, and Yann LeCun. What drives success in physical planning with joint-embedding predictive world models? *Transactions on Machine Learning Research*, 2026. URL https://arxiv.org/abs/2512.24497.

Yi Wan, Muhammad Zaheer, Adam White, Martha White, and Richard S. Sutton. Planning with expectation models. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 3649–3655, 2019.

Haodong Yan, Jiaguan Zhu, Mingyuan Jia, Ruiqing Yin, Junjie He, Zhide Zhong, Junfeng Li, Jinxuan Lu, Hengtao Li, Tianran Zhang, Jiayi Chen, Wenxuan Song, Wen Chen, Yuxiang Gao, and Haoang Li. Is forward prediction enough? physical state grounding for JEPA world models, 2026. URL https://arxiv.org/abs/2608.06799.

Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning, 2024.

A What the objective does not guarantee

The experiments concern fitted finite networks. The following population statements ask a narrower question: which guarantees follow from prediction error and a Gaussian latent marginal alone? They do not characterize optimization or identify the cause of the empirical failures.

A.1 Non-collapse does not select the controlled state

Write observations as $x_t = R(p_t, b)$, where p_t is controlled state and b is an episode-constant nuisance recoverable from the image. Assume that b is independent of states and actions in the training data. At deployment, assume it is independent of the initial state and transition randomness, and does not directly affect dynamics or reward. Let SIGReg_∞ be the nonnegative population characteristic-function discrepancy, which vanishes for $\mathcal{N}(0, I_D)$ (Appendix B).

Proposition 1 (Gaussian nuisance optimum; extending Sobal et al. (2022))**.** *If the encoder class contains a map $f(R(p, b)) = \varphi(b)$ with $\varphi(b) \sim \mathcal{N}(0, I_D)$ and the predictor class contains the last-latent copy map, their population prediction loss and SIGReg_∞ are both zero. This is a global optimum with covariance I_D and $I(z_t; p_{1:T}) = 0$ under the training distribution. A deployed controller observing only these latents, time, and its own actions has the value of a randomized state-blind policy.*

The argument is simple: nuisance changes across episodes, satisfying the marginal constraint, but is constant within each episode, making prediction exact. Its deployment value follows by giving a state-blind policy the same private nuisance draw and controller randomness. The Gaussianization assumption requires a rich nuisance and a realizable measurable map; a finite texture does not literally instantiate it. The finite-batch regularizer also differs from the population functional. A complementary innovation bound in Appendix B shows that an encoder preserving all state distances at a fixed scale must incur prediction error for unpredictable state innovations.

A.2 Decodability does not set a useful prediction scale

Even preserving all state information leaves its geometry unconstrained. For any $\varepsilon > 0$, we can place the two values of a relevant bit less than 2ε apart while keeping the *entire* latent distribution exactly standard Gaussian.

Proposition 2 (Exact information at arbitrarily small scale)**.** *For every $\varepsilon > 0$ and $D \geq 1$, there is a measurable observation process with episode-constant nuisance b and a fresh independent fair state bit p_t, and an encoder f_ε, such that $z_t \sim \mathcal{N}(0, I_D)$ and both p_t and b are exactly recoverable almost surely. For fixed b, $0 < \|f_\varepsilon(R(1,b)) - f_\varepsilon(R(0,b))\| < 2\varepsilon$. The optimal next-latent predictor is independent of the next bit and has coordinate-mean squared error in $(0, \varepsilon^2/D]$. With reward $\mathbf{1}\{a_t = p_t\}$, latent feedback attains value 1 per step, while every state-blind policy attains $1/2$.*

The construction partitions one Gaussian coordinate into narrow intervals and uses the bit to select the lower or upper half of each interval's conditional probability mass. The nuisance fixes the interval and the within-half quantile. This preserves the Gaussian marginal and exact decoding while making the two states arbitrarily close. The full proof is in Appendix B. It uses infinite-precision measurable coding; the next bit is intrinsically unpredictable. It therefore separates information from scale, without proving avoidable prediction error or explaining deletion of the predictable gate wall.

Why scale matters for a downstream read-out. For any fixed L-Lipschitz downstream score q (for example, an action-value read-out) and any coupled real and predicted next latents,

$$\left|\mathbb{E}q(Z') - \mathbb{E}q(\widehat{Z}')\right| \leq L\sqrt{\mathbb{E}\|Z' - \widehat{Z}'\|^2}. \tag{4}$$

This standard bound follows from Lipschitz continuity and Cauchy–Schwarz. Prediction error controls score error only relative to the read-out's sensitivity. Exact decodability alone supplies no bound on that sensitivity. We do not estimate L or claim a control guarantee for the fitted networks; the inequality makes explicit the missing bridge between a geometric loss and a decision.

B Proofs

B.1 Empirical objective and population idealization

For a training batch of B windows and H predicted time positions, the implemented loss is

$$\mathcal{L}_{\text{pred}} = \frac{1}{BHD}\sum_{b,h}\|\widehat{z}_{b,h+1} - z_{b,h+1}\|^2. \tag{5}$$

The notation in Equation 1 averages over time and uses this coordinate-mean convention. One-step targets are teacher-forced at every temporal position, with a shorter history near the start of a window. For encoded windows of T frames, P random unit projection directions u_p, K knots $\tau_k \in [0, 3]$, and $w_k = e^{-\tau_k^2/2}$, the regularizer is

$$R = \frac{B}{TP}\sum_{t,p,k} q_k w_k \left[\left(\frac{1}{B}\sum_b \cos(\tau_k u_p^\top z_{b,t}) - w_k\right)^2 + \left(\frac{1}{B}\sum_b \sin(\tau_k u_p^\top z_{b,t})\right)^2\right], \tag{6}$$

where $\Delta = 3/(K-1)$, with $q_k = \Delta$ at the endpoints and 2Δ in the interior. The characteristic function is averaged across batch examples separately at each temporal position; the factor B is part of the implementation with $\lambda = 0.09$.

The population statements use the nonnegative discrepancy

$$\text{SIGReg}_\infty(\mathcal{L}(z)) = \int_{\mathbb{S}^{D-1}}\int_0^3 w(\tau)\left[\left(\mathbb{E}\cos(\tau\langle z, u\rangle) - w(\tau)\right)^2 + \left(\mathbb{E}\sin(\tau\langle z, u\rangle)\right)^2\right] d\tau\, d\sigma(u), \tag{7}$$

where $w(\tau) = e^{-\tau^2/2}$ and σ is uniform on the sphere. Only the fact that this functional vanishes for a standard Gaussian is needed. It is not a theorem about exact minima of the finite-batch objective.

B.2 NUISANCE OPTIMUM AND THE COST OF INNOVATIONS

Proposition 1.

Proof. Let β recover the nuisance from the observation, set $f^\star = \varphi \circ \beta$, and let $g^\star$ copy the last latent. Then $z_t = \varphi(b) = z_{t+1}$, so every prediction residual vanishes. The Gaussian marginal gives covariance I_D and zero population regularizer. Both objective terms are nonnegative, so their sum is globally minimized at zero. Training-distribution independence gives $I(\varphi(b); p_{1:T}) = 0$.

For deployment, draw nuisance B and the controller's random seed U at reset, independently of the initial state and transition randomness. Run the original controller in the environment with nuisance B. Let a state-blind policy with private seed (B, U) feed constant latent history $\varphi(B)$ to a copy of that controller and reproduce its actions with U. Coupled to the same initial state and transition randomness, the policies take the same first action. If their actions agree through step t, their states, supplied latent histories, and seeds agree, so the next actions agree as well. Induction gives identical trajectories, returns, and terminations. This does not assume independence between B and states subsequently influenced by its induced actions. □

Corollary 1 (Innovation cost for uniformly state-resolving encoders)**.** *Let $p_{t+1} = p_t + \mu(a_t) + w_t$, $\mathcal{H}_t = \sigma(p_{\le t}, b, a_{\le t})$, and $\Sigma_t = \mathrm{Cov}(w_t \mid \mathcal{H}_t)$. Assume finite second moments of states and encoded observations, $x_t = R(p_t, b)$, fixed nuisance b, and $\|f(R(p, b)) - f(R(p', b))\| \ge \kappa \|p - p'\|$ for all admissible p, p', b, with $\kappa > 0$. Every predictor of latent and action history satisfies*

$$\frac{1}{D}\mathbb{E}\|g - f(x_{t+1})\|^2 \ge \frac{\kappa^2}{D}\mathbb{E}\mathrm{tr}\Sigma_t. \tag{8}$$

When $\mathbb{E}\mathrm{tr}\Sigma_t > 0$, this exceeds the nuisance encoder's zero prediction loss under the assumptions of Proposition 1.

The assumption is stronger than injectivity or task sufficiency: every state distance must be preserved at a uniform scale. Innovation is measured after conditioning on state history and nuisance; persistent wind is not automatically fresh innovation.

Corollary 1.

Proof. Put $Z = f(x_{t+1})$ and $\mathcal{C}_t = \sigma(z_{\le t}, a_{\le t})$. Because observations and latents are measurable functions of state and nuisance, $\mathcal{C}_t \subseteq \mathcal{H}_t$. Any predictor using a shorter window is also $\mathcal{C}_t$-measurable. Conditional least squares and the tower property give

$$\begin{aligned}\mathbb{E}\|g - Z\|^2 &\ge \mathbb{E}\|Z - \mathbb{E}[Z \mid \mathcal{C}_t]\|^2 \\ &= \mathbb{E}\|Z - \mathbb{E}[Z \mid \mathcal{H}_t]\|^2 + \mathbb{E}\|\mathbb{E}[Z \mid \mathcal{H}_t] - \mathbb{E}[Z \mid \mathcal{C}_t]\|^2 \\ &\ge \mathbb{E}\mathrm{trCov}(Z \mid \mathcal{H}_t).\end{aligned}$$

The cross term vanishes because its first factor has conditional mean zero given $\mathcal{H}_t$. Take conditionally independent copies P, P' of p_{t+1} given $\mathcal{H}_t$. Such copies exist for Euclidean state spaces, using their regular conditional distributions. The nuisance b and current state p_t are fixed under this conditioning. The conditional independent-copy variance identity yields

$$\begin{aligned}\mathrm{trCov}(Z \mid \mathcal{H}_t) &= \tfrac{1}{2}\mathbb{E}[\|f(R(P, b)) - f(R(P', b))\|^2 \mid \mathcal{H}_t] \\ &\ge \tfrac{\kappa^2}{2}\mathbb{E}[\|P - P'\|^2 \mid \mathcal{H}_t] \\ &= \kappa^2 \mathrm{trCov}(p_{t+1} \mid \mathcal{H}_t) = \kappa^2 \mathrm{tr}\Sigma_t.\end{aligned}$$

The final equality follows because $p_t + \mu(a_t)$ is $\mathcal{H}_t$-measurable. Taking expectations and dividing by D proves Equation 8. Averaging these inequalities gives the corresponding bound for a multi-time prediction objective. Proposition 1 supplies the zero-loss comparator when its assumptions also hold. □

B.3 CONDITIONAL PREDICTION AND GEOMETRIC SCALE

Lemma 1 (Conditional mean)**.** *For a fixed encoder and data law with $\mathbb{E}\|z'\|^2 < \infty$, the unique squared-error minimizer over measurable square-integrable predictors from history c and action a is $m(c, a) = \mathbb{E}[z' \mid c, a]$, up to null sets.*

Proof. Let $m = \mathbb{E}[z' \mid c, a]$. For every square-integrable measurable $g(c, a)$, expand $g - z' = (g - m) + (m - z')$. The cross term has expectation zero since $\mathbb{E}[m - z' \mid c, a] = 0$. Hence

$$\mathbb{E}\|g - z'\|^2 = \mathbb{E}\|g - m\|^2 + \mathbb{E}\mathrm{trCov}(z' \mid c, a).$$

The second term does not depend on g. The first is nonnegative and vanishes exactly when $g = m$ almost surely. □

The finite-sample regularizer. The finite-sample regularizer differs from Equation 7. For independent identically distributed samples with characteristic function ϕ and empirical estimate $\widehat{\phi}_n$,

$$\mathbb{E}|\widehat{\phi}_n(t) - w(t)|^2 = |\phi(t) - w(t)|^2 + \frac{1 - |\phi(t)|^2}{n}. \tag{9}$$

Indeed, $\mathbb{E}\widehat{\phi}_n = \phi$ and $\mathbb{E}|\widehat{\phi}_n - \phi|^2 = (1 - |\phi|^2)/n$, since each complex exponential has modulus one; expanding around ϕ gives the identity. At a Gaussian law, the residual equals $(1 - e^{-t^2})/n$. This is estimator bias, not an optimization floor. Changing the law changes both terms; dependent temporal samples introduce additional covariance terms. The implementation multiplies this statistic by batch size B, so its Gaussian finite-batch bias does not shrink as $1/B$ after that scaling. Finite projections and knots also do not characterize Gaussianity.

Proposition 2.

Proof. Partition $\mathbb{R}$ into $I_j = [2\varepsilon j, 2\varepsilon(j + 1))$, $j \in \mathbb{Z}$. Let $q_j = \Pr(G \in I_j)$ for $G \sim \mathcal{N}(0, 1)$. Every q_j is positive and their sum is one. Let F_j be the conditional Gaussian distribution function on I_j, with inverse quantile function Q_j on $(0, 1)$. The conditional CDF is continuous and strictly increasing on the interval interior, so its inverse is measurable and takes values in that interior.

Draw J with probabilities q_j, $U \sim \mathrm{Uniform}(0, 1)$, and $V \sim \mathcal{N}(0, I_{D-1})$, independently. Set $b = (J, U, V)$; for $D = 1$, omit V. Take $R(p, b) = (p, b)$ and define

$$f_\varepsilon(R(p, b)) = \big(Q_J((U + p)/2), V\big). \tag{10}$$

The mixture $(U + p_t)/2$ is uniform on $(0, 1)$, since p_t is an independent fair bit. Conditional on $J = j$, the first coordinate therefore has the Gaussian law restricted to I_j. Mixing with weights q_j recovers $\mathcal{N}(0, 1)$. Independence of V gives the joint law $\mathcal{N}(0, I_D)$ and zero population regularizer.

Given the first coordinate y, its interval determines J almost surely. Write $v = F_J(y)$. Then $p_t = \mathbf{1}\{v > 1/2\}$ and $U = 2v - p_t$ almost surely; the other coordinates recover V. Thus the latent recovers both state and nuisance. For fixed b, the two first coordinates are distinct and lie inside the same interval of width 2ε. All remaining coordinates agree, proving the distance bound.

Latent history reveals b, while the next bit remains independent and fair. The next latent consequently has two equiprobable values, say $z^{(0)}(b)$ and $z^{(1)}(b)$. Lemma 1 gives $m(b) = (z^{(0)}(b) + z^{(1)}(b))/2$. Its conditional squared error is $\|z^{(1)}(b) - z^{(0)}(b)\|^2/4$, strictly positive and bounded by ε^2. The midpoint is square-integrable by Jensen's inequality, since both state encodings have finite second moments. Taking expectations and dividing by D proves the coordinate-mean loss bound, including strict positivity since the conditional error is positive almost surely. The midpoint depends only on b, so it is independent of p_{t+1}. Finally, decoding the current bit permits $a_t = p_t$. A state-blind action is independent of the current fair bit, giving success probability $1/2$. □

C THE CUBE WORLDS

The position-readout study crosses black, per-frame random, and per-episode random backgrounds with deterministic or hidden-wind dynamics. Color-noise and black-background datasets contain 100,000 transitions; the gray-noise conditions contain 500,000. Only the four gray grid-JEPA conditions have three independent model initializations. Table 1 and Figure 5 report that replicated comparison. Tables 4 and 5 retain seed-0 outcomes for the broader conditions, using the same canonical benchmark files throughout. Differences across dataset sizes and model recipes are not isolated nuisance effects.

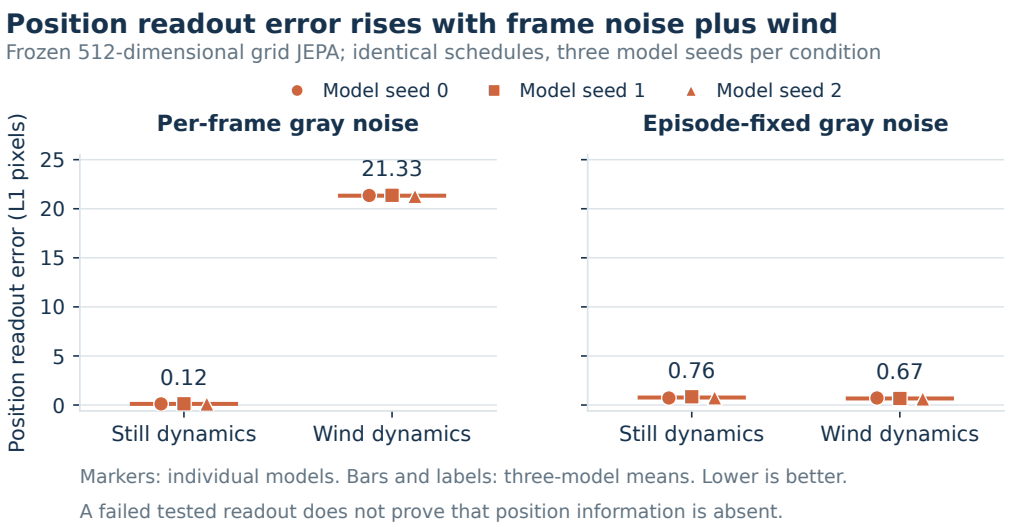


Figure 5: **Same objective, same architecture, same data: the cube is kept or lost depending on how the nuisance and the dynamics interact.** Each marker is one fitted model; bars are three-model means. A failed read-out does not prove that no decoder could recover the position; it shows that the fixed diagnostic that succeeds in three cells fails in the fourth.

Environment. The scene contains a red 6×6 object in a 63×63 RGB image with a red boundary. Four cardinal actions move the object three pixels. Reset positions are uniformly sampled from integer coordinates in $[16, 42]^2$, and initial wind is zero. In wind conditions, each wind coordinate independently increments by -1, 0, or 1 with probabilities 0.1, 0.8, and 0.1, clipped to $[-2, 2]$; the updated wind is added to the action displacement. Background pixels are uniform integers from 0 to 255; gray noise shares the same sample across RGB channels. Contact with the boundary gives reward -1 and terminates; other rewards are zero, with a 128-step cap.

Read-out fitting. The nonlinear latent position probe is fitted on training data and evaluated on independent `eval.npz` frames. Its spatial softmax-and-offset output is produced by two hidden layers of width 128, trained for 10,000 updates with batch 2,048 and Adam at initial learning rate 10^{-3} with cosine decay. Grid inputs are standardized using training statistics. For class-token inputs, standardization and PCA whitening are selected using the final 10% of fitting rows, then the chosen variant is refitted on the full fitting split. Whitening floors singular values at 1% of the largest. Applying this read-out to predicted latents tests both prediction and read-out extrapolation. The pixel read-out used for decoded flow images is a separate supervised CNN trained with synthetic background replacement using ground-truth cube masks; it is a diagnostic, not a policy or a world-model training signal. Rollout windows are sampled with replacement and overlap, and exclude death or reset during the eight context and 16 forecast frames. The configured benchmark uses 2,048 windows for video metrics and the first 512 for position diagnostics; exact sampled indices were not retained in the archived result files. Surviving windows are a selected subset of wind trajectories, so these errors do not establish death prediction or closed-loop control.

Earlier controller experiments use the three black/color background conditions. Their final survival outcomes are retained below. The class-token JEPA's near-zero loss on per-episode texture and poor reward-driven control are consistent with the known predictable-nuisance mechanism, without exactly realizing Proposition 1. The replicated gray-frame/wind failure instead has high prediction loss and should not be interpreted as the same mechanism. Still-world survival can be achieved by state-blind alternating actions, so it does not certify state feedback.

Table 4: Seed-0 cube position L1 error (pixels) for the grid and class-token JEPAs. "Real" evaluates held-out encoded frames; "Pred. 8" evaluates the same read-out on eight-step predicted latents. Gray conditions use more training transitions than black and color conditions.

Background	Wind	Grid JEPA		Class-token JEPA	
		Real	Pred. 8	Real	Pred. 8
Black	Off	0.097	3.933	0.087	45.268
	On	0.126	11.732	0.049	8.957
Color, frame	Off	0.172	5.570	24.133	19.428
	On	0.253	11.823	25.185	19.317
Color, episode	Off	3.273	6.594	20.804	21.567
	On	1.451	13.342	23.431	21.807
Gray, frame	Off	0.115	4.269	23.954	20.286
	On	21.354	19.696	25.288	19.567
Gray, episode	Off	0.715	1.779	21.316	21.284
	On	0.717	11.553	23.880	23.386

Table 5: Seed-0 flow cube position L1 error (pixels). Latent and pixel columns use different supervised read-outs. The persistent-color failure is visible before dynamics: real encoded position has large error. These are model-recipe references rather than objective-only comparisons.

Background	Wind	Real latent	Pred. 8 latent	Pred. 8 pixels
Black	Off	0.140	0.126	0.242
	On	0.153	11.632	11.622
Color, frame	Off	1.088	1.059	0.395
	On	1.344	12.311	11.985
Color, episode	Off	18.563	21.954	22.423
	On	7.010	18.960	16.937
Gray, frame	Off	3.114	2.667	0.382
	On	3.690	14.392	12.302
Gray, episode	Off	5.062	4.896	1.339
	On	3.066	15.150	13.583

Table 6: Survival (percent) in the supplementary cube worlds. "LeWM" denotes the class-token JEPA; "grid" denotes the grid JEPA; "goal" and "heads" use latent-goal and learned-return search, respectively. PPO columns use the final saved policies; "px" and "z" identify pixel and latent observations. Still-world survival does not certify state feedback.

Background	Black		Per-frame noise		Per-episode noise	
Dynamics	Still	Wind	Still	Wind	Still	Wind
random	20	0	20	0	20	0
centre	100	99	100	99	100	99
LeWM goal	97	64	5	0	80	0
LeWM heads	100	60	9	0	2	0
LeWM PPO	100	99	0	0	0	0
grid goal	88	1	92	0	100	77
grid heads	100	5	100	0	85	8
grid PPO	100	94	100	0	2	0
flow PPO px	100	94	98	97	98	95
flow PPO z	100	96	99	1	51	1
PPO real	100	98	100	96	100	78

D HYPERPARAMETERS AND COMPUTE

The gate and gray-noise cube conditions supply 500,000 uniform-random transitions each. The earlier black and color-noise cube datasets contain 100,000 transitions. Craftax uses its separate 500,000-transition archive. Model training uses bfloat16 compute, float32 parameters, and one A100. A 500-step calibration estimates the training length fitting 575 seconds including compilation; the grid gate model trains for 8,500 steps (17,000 and 34,000 in the earlier budget extensions of Appendix L), the class-token cube models for 23,750–24,000. Model seeds 0, 1, 2 of the grid JEPA differ only in the initialisation and data-order seed.

JEPA settings. Both JEPA architectures use batch 128 and AdamW with learning rate and weight decay 10^{-3}, 10% warm-up followed by cosine decay to zero, and gradient clipping at 1.0. SIGReg uses weight 0.09, 17 knots on $[0, 3]$, and 1024 random projections. Neither architecture uses a stop-gradient, teacher, decoder, or reward during world-model fitting. Linear–BatchNorm–GELU–linear projection heads flatten batch, time, and spatial tokens for normalization. Training-mode statistics therefore couple examples and temporal positions; at inference, running statistics define a fixed pointwise projection and the attention predictor is causal. The theoretical causal-map formulation is an inference/population idealization.

	Class-token JEPA	Grid JEPA
encoder	ViT, 7×7 patches	tile transformer + conv
width / depth / heads	192 / 6 / 3	128 / 6 / 8
encoder MLP	768	$4\times$ width
read-out	class token	4×4 grid, 32 channels
latent dimension	192	512
projector	BN projector, width 2048	same, hidden 512
predictor	AdaLN-zero AR	spatial + causal temporal
predictor depth / heads / dim	6 / 16 / 64	6 / 8 / 32
predictor MLP / width	2048 / latent	$2\times$ width / 256
history	3 frames	8 frames
dropout	0.1	0
parameters	15.3M	2.6M + 8.3M

Flow settings. The tokenizer uses 9,000 steps, batch 256, and learning rate 9×10^{-4}; dynamics 16,000 steps, batch 64, the same learning rate, width 256, six layers, eight spatial and temporal attention heads, a 32-frame context, independent per-frame noise levels, a shortcut loss of weight 0.25 every fourth step after the first quarter of training, history dropout 0.25, two history frames resampled every eight steps after half of training, and a temporal loss weight of 0.25. The symlog reward head has 255 bins over $[-20, 20]$; reward and termination losses have weight 0.1 and enter the trunk at full strength for half of training, then at factor 0.1. Inference uses four function evaluations per frame. We corrected terminal-transition masking in the recipe's reward loss (Appendix H).

JEPA read-outs. Gate post-hoc heads use the frozen encoder and the first 800 time rows (80,000 frames) of the same archive used for world-model training. The following 200 time rows are held out from head fitting only; the independent world-model evaluation archive is `eval.npz`. The pixel decoder uses 8,000 steps, batch 256, learning rate 10^{-3}, width 256, six residual blocks; it supports the figures and the decoded-latent control, in which its frozen output, clipped to $[0, 255]$, replaces the pixels of the real-gate PPO; no true pixel enters that policy after the encoder. Reward, termination, and behaviour-value heads are two-layer perceptrons of width 512 on the latent and the one-hot action, 6,000 steps, batch 1024, learning rate 10^{-3}, Monte Carlo value targets at discount 0.99, value loss weight 0.1. The residual predictor models $z_{t+1} - m$ with $m = g(z_{t-7:t}, a_{t-7:t})$ the original grid prediction, conditioned on m, z_t, and the action: three residual blocks of width 1024, 8,000 steps, and batch 2048. The final 20,000 valid windows of the same world-model training archive are excluded from residual fitting and used for its quality check. They are not independent world-model evaluation data. For encoder seed 0, coordinate-mean MSE is 3.07×10^{-4} for the original predictor, 2.05×10^{-4} for one residual sample, and 1.32×10^{-4} for the mean of eight. This

quality check uses eight function evaluations per sample; the reported PPO runs instead use four. Averaging eight learned samples is a random Monte Carlo average, not an exact conditional mean.

Policy optimisation (historical protocols). The settings in this paragraph are those of the earlier gate and cube studies of Appendices L and M; the shared-policy comparison of the main text uses Appendix J and the Craftax comparison Appendix K. PPO uses discount 0.99, GAE parameter 0.9, clipping 0.2, value coefficient 0.5, entropy coefficient 0.01, AdamW at 2.5×10^{-4} with weight decay 0.01, four epochs per batch, and gradient clipping at 1.0. Pixels pass through a three-block Impala torso (Espeholt et al., 2018), latents through a two-layer perceptron; a GRU receives that embedding and the previous action; the critic uses 64 categorical bins with histogram-Gaussian targets (Farebrother et al., 2024). Real-environment PPO uses 128 environments and 64 steps per iteration with four minibatch groups, truncated backpropagation over 16 steps, and 1.5M environment steps. In-model PPO uses 150 updates of 512 trajectories of 16 steps (1.2M imagined steps) with minibatch size 128; half of the starts are resets, half dataset states with their real eight-frame context. All reported policy rows use final checkpoints.

Compute. On a shared A100, the seed-0 grid-gate JEPA completes 8,500 updates in 573 seconds and has a logged total of 586.5 seconds. Probe and head fitting takes approximately three minutes. The residual sampler's optimization stage takes 74.3 seconds; model loading, latent extraction, fitting, and quality evaluation together take 275.2 seconds before the final file write. Other logged stages are PPO inside the JEPA 2.0–4.3 min per run; PPO inside the flow model 5–6 min per run at four function evaluations, 20–26 min with the mean of eight; PPO in the real environment 3.7–7.7 min on pixels with the convolutional torso and 51–70 s with the MLP torso, 76–78 s on the frozen latent, 4.7–4.8 min on the decoded latent with the convolutional torso and 1.8–2.9 min with the MLP torso; the latent diagnostic: 2 min.

E Evaluation protocol

Every gate-policy endpoint uses the final checkpoint and 256 real episodes from reset, with the 128-step cap. Survival is the fraction still alive after step 128; death on that step is not survival. Walls passed counts wall exits before or on the terminal transition, including an exit on the same step as death. Discovery is final survival strictly above 50%. Real-training and imagined-training runs advance their random keys under different iteration schedules; their final evaluations are not paired episode trials. Policy seeds are separate runs conditional on a fitted model; independently fitted encoders provide a different replication level. The flow model has only one checkpoint. Reported means weight policy runs equally; standard deviations describe variation across those runs, not uncertainty across model families.

Gate specification. The gate uses the same image and object sizes and cardinal action displacement. A white vertical wall of width three pixels spawns at $x = 60$ every 22 steps and moves left three pixels per step. Its 16-pixel gap is sampled at spawn, top or bottom with equal probability, and becomes visible when the wall reaches $x \leq 40$. The gap location is hidden state before that reveal. Border or wall contact produces reward -1 and termination; the time cap is 128 steps.

Latent diagnostic. For each encoder, the diagnostic uses 64 real contexts with the wall still solid at $x \in \{42, 45\}$. Reconstructed simulator states generate recorded futures and matched wall-free futures under the recorded actions. The recorded branch survives the diagnostic horizon by construction. At each step, wall amplitude is the squared Euclidean distance between encodings of the wall-present and wall-absent frames. The teacher-forced residual uses real encoded history and is a total squared error, summed over latent coordinates. The wall coordinate of a prediction is its projection onto the wall-present minus wall-absent latent difference, with endpoints 0 and 1. Decoded wall intensity averages the minimum colour channel on shared wall pixels, excluding either possible gap band. These measurements compare features within one encoder; raw distances across models are not an information metric.

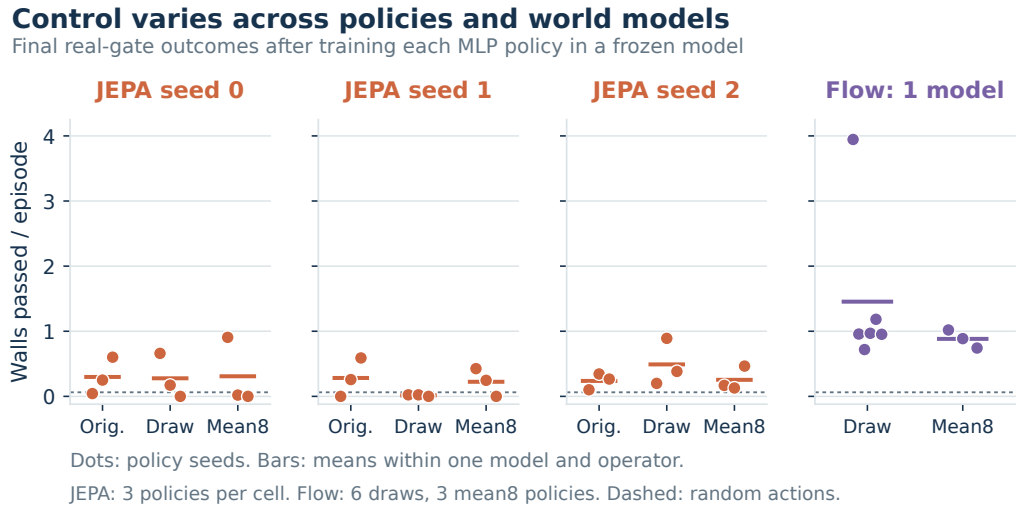


Figure 6: **Control outcomes by model fit and predictor variant.** Each point is a final policy evaluated on 256 real episodes. Original prediction, residual sampling, and residual means are separate conditions; model and policy replication are distinct. Horizontal bars show within-model means, and the dashed line marks random actions. The flow reference uses one checkpoint.

F LATENT-POLICY TORSO CONTROLS

The MLP on decoded latents passes 0.92 walls per episode with no discoveries in six runs (Table 7), using encoders 0 and 1. The decoded CNN control instead uses all three encoders, with three policies per encoder. Both of its discoveries use encoder 1. The decoder is frozen and reconstruction-supervised; success therefore establishes useful encoded information under this added read-out, not sufficiency of the unsupervised training objective. Failure of the true-pixel MLP recipe establishes no upper bound on MLP capacity.

Table 7: Real-gate observation and torso controls: final walls per episode and survival, mean ± standard deviation over policies; discovery means final survival above 50%. All policies use 1,499,136 real transitions.

observation of the real-gate PPO	runs	walls passed	survival (%)	discovered
pixels, convolutional torso	3	2.91±1.65	65±52	2/3
pixels, MLP torso	3	0.91±0.08	4±2	0/3
frozen JEPA latent, MLP torso	9	0.94±0.04	4±1	0/9
decoded JEPA latent, convolutional torso	9	1.57±1.26	22±37	2/9
decoded JEPA latent, MLP torso	6	0.92±0.08	3±2	0/6

G CONTROL OUTCOMES AND LATENT DIAGNOSTICS

Table 8 gives aggregate final outcomes; Table 9 lists every imagined JEPA policy. The pooled imagined JEPA mean weights all original, sampled-residual and mean-residual runs equally. The flow rows distinguish six sampled policies from three policies using a mean of eight, all sharing the same model fit. They do not estimate variation across independent flow-model fits. Tables 10 and 11 retain the encoder and geometry diagnostics supporting the gate observations.

Table 8: Final control outcomes in the real gate, mean ± standard deviation over policy runs. Imagined training uses 1,228,800 transitions per policy; real training uses 1,499,136. Model-fit counts and policy-run counts identify distinct replication levels.

Controller	Models	Policies	Walls	Survival (%)	Discovery
JEPA: original	3	9	0.27 ± 0.22	0 ± 1	0/9
JEPA: mean of 8	3	9	0.26 ± 0.30	1 ± 1	0/9
JEPA: one sample	3	9	0.26 ± 0.32	1 ± 1	0/9
Flow: mean of 8	1	3	0.88 ± 0.14	3 ± 1	0/3
Flow: one sample	1	6	1.45 ± 1.23	19 ± 39	1/6
Random	–	–	0.06	0	–
Alternating	–	–	0.02	0	–
Real pixels: CNN	–	3	2.91 ± 1.65	65 ± 52	2/3
Real pixels: MLP	–	3	0.91 ± 0.08	4 ± 2	0/3
Real JEPA: MLP	3	9	0.94 ± 0.04	4 ± 1	0/9
Real decoded: CNN	3	9	1.57 ± 1.26	22 ± 37	2/9

Table 9: Every imagined JEPA policy at its final checkpoint: survival (percent) and walls per episode by encoder seed, predictor variant and PPO seed. Each cell uses 256 real episodes.

		Survival (%)			Walls passed		
Predictor	Model	PPO 0	PPO 1	PPO 2	PPO 0	PPO 1	PPO 2
Original	0	0	0	1	0.04	0.25	0.60
	1	0	0	1	0.00	0.26	0.59
	2	0	1	0	0.10	0.34	0.27
Residual mean	0	3	0	0	0.91	0.02	0.00
	1	2	1	0	0.43	0.25	0.00
	2	0	0	1	0.17	0.13	0.46
Residual sample	0	2	1	0	0.66	0.17	0.00
	1	0	0	0	0.02	0.02	0.00
	2	0	3	1	0.20	0.89	0.38

Table 10: Grid-JEPA diagnostics by encoder seed. Wall amplitude and total teacher-forced residual are squared distances at future steps 1 and 8; decoded intensity uses steps 1 and 2. The wall coordinate has endpoints 0 (wall absent) and 1 (true encoding). Head AUC compares each death cause with alive transitions; PPO wall counts pool predictor variants.

JEPA encoder seed	0	1	2
Wall amplitude, step 1	0.035	0.028	0.088
Wall amplitude, step 8	0.124	0.082	0.193
Total residual, step 1	0.145	0.130	0.156
Total residual, step 8	0.249	0.228	0.279
Wall coordinate, step 1	0.39	0.45	0.66
Decoded wall, step 1	0.00	0.00	0.68
Decoded wall, step 2	0.00	0.00	0.02
Border-death AUC	0.96	0.96	0.88
Wall-death AUC	0.65	0.65	0.76
Imagined PPO, walls	0.29±0.34	0.17±0.22	0.33±0.24

Table 11: JEPA geometry and decoded wall intensity for model seed 0, averaged over 64 pre-reveal contexts. Distances use matched actions and the same encoder; total residuals include all latent coordinates. Wall intensity is measured on shared wall pixels.

Future step	1	3	5	8
Squared distance, summed over coordinates				
Wall-present versus wall-absent	0.035	0.041	0.071	0.124
Total teacher-forced residual	0.145	0.152	0.200	0.249
Residual along wall direction	0.015	0.021	0.048	0.104
Residual orthogonal to wall direction	0.130	0.132	0.152	0.145
Predicted wall coordinate	0.39	0.33	0.29	0.21
Decoded wall intensity				
Encoded true future	1.00	1.00	1.00	1.00
Original JEPA rollout	0.00	0.00	0.00	0.00

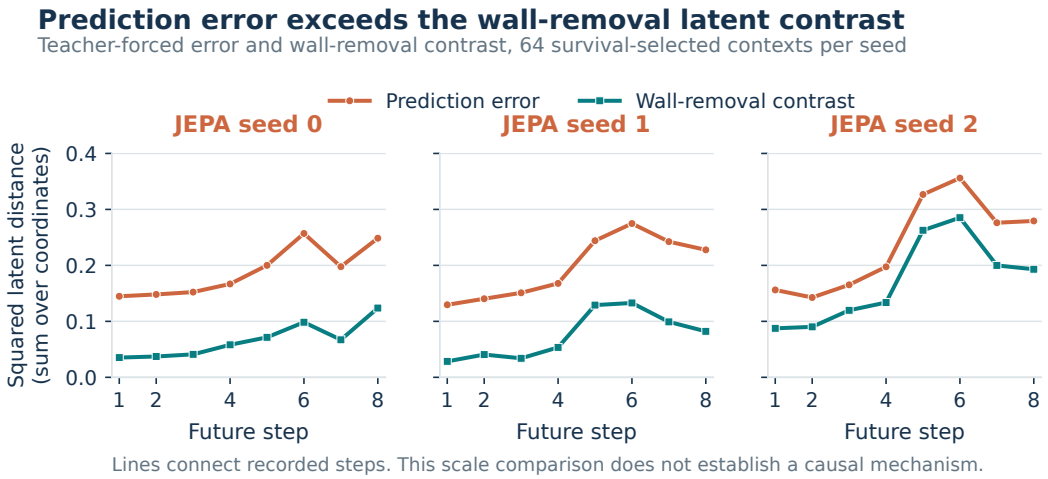


Figure 7: **Within-encoder scale diagnostics.** For each of the three gate encoders, wall-removal distance and teacher-forced total one-step residual are summed squared latent distances. Curves average 64 pre-reveal contexts whose recorded futures survive the diagnostic horizon. These are feature sensitivity and total prediction error, not commensurate measures of information content.

H TERMINAL REWARDS IN THE FLOW RECIPE

The original recipe masks transitions entering reset frames out of reward training. Here the only nonzero reward occurs on terminal transitions, so that mask removes the reward signal. The corrected head predicts the next reward from the current frame summary and action with terminal transitions retained in its loss. The reported flow-policy outcomes use this corrected convention. The single-checkpoint reference therefore denotes the complete tokenizer, dynamics, reward and termination package, rather than an isolated change in transition stochasticity.

I CRAFTAX PROTOCOLS AND SINGLE-CHECKPOINT RESULTS

I.1 FRESH LOCAL-GOAL EVALUATION

The new evaluation uses the same existing 512-dimensional checkpoint, model seed 0, with 128 starts for each evaluation seed 101, 202, and 303. Goal generation and real execution have separate pseudorandom streams; all controllers share initial states and the execution stream. The target is the endpoint after six random actions, following seven warm-up actions from reset. The objective is final player-position equality with survival throughout the six executed actions. Every evaluated trajectory survives, so the survival requirement does not alter the measured rates. The target position displacements are zero, one, two, or three grid cells in 137, 148, 84, and 15 cases, respectively.

The open-loop planner solves once at horizon six. Receding-horizon planning receives each real observation, executes the first selected action, and replans over the remaining horizons $6, 5, 4, 3, 2, 1$. Every CEM solve uses 512 candidate sequences, 32 elites, and six iterations. Thus receding planning uses six times as many candidate-sequence searches and 3.5 times as many predicted latent transitions as one-shot planning, excluding additional encoding. It is not a compute-matched feedback intervention. Both controllers use terminal latent MSE and the native 17 actions, without reward heads, decoder use, latent rescaling, or privileged coordinates in the planning cost.

Table 12: Craftax goal-success counts and Wilson 95% intervals, conditional on one fitted model. The reference replays the privileged sequence used to generate the goal under independent execution noise; it is a protocol check, not an oracle or a guaranteed upper bound.

Controller	Success / total	Rate (%)	Wilson interval (%)
No-op	$137/384$	35.68	$[31.05, 40.59]$
Random	$65/384$	16.93	$[13.51, 21.00]$
JEPA open loop	$358/384$	93.23	$[90.26, 95.34]$
JEPA receding horizon	$369/384$	96.09	$[93.66, 97.62]$
Reference-action replay	$382/384$	99.48	$[98.12, 99.86]$

Open-loop and receding-horizon success on displaced targets is $222/247$ and $232/247$; for displacements of at least two cells it is $80/99$ and $87/99$. Random actions reach $19/247$ and $1/99$ in these strata, while no-op reaches neither. Mean endpoint position L1 errors are 0.1068 open loop, 0.0781 receding horizon, 1.5599 random, and 0.9401 no-op, in grid-cell units. Receding-horizon success exceeds open loop by 2.86 percentage points; a paired episode bootstrap with 10,000 resamples gives $[0.78, 4.95]$ percentage points. All intervals quantify evaluation variation for this fixed checkpoint. The three evaluation seeds are not training replications.

Table 13: Craftax success by target displacement. The first column includes all 384 starts; the others retain 247 displaced targets and 99 targets at least two grid cells away.

Controller	All (384)	Displaced (247)	≥ 2 cells (99)
No-op	35.7%	0.0%	0.0%
Random	16.9%	7.7%	1.0%
JEPA open loop	93.2%	89.9%	80.8%
JEPA receding horizon	96.1%	93.9%	87.9%

I.2 Earlier exploratory goal and reward protocols

The earlier results below use different evaluation episodes and a shared goal-generation/execution noise protocol. They are not pooled with the fresh independent-noise evaluation above.

The grid checkpoint uses training seed 0, a tile encoder, $16 \times 32 = 512$ latent coordinates, a six-layer factored predictor of width 256, history length eight, 17 actions, batch 128, and 8,500 updates. The recorded parameter count is 11.12 million. Training takes 604.5 seconds from loading through the logged end-to-end timer (578.5 seconds in the update loop); thus it is an approximately ten-minute run, not a strict sub-600-second result. The frozen decoder and heads add 175.6 seconds. Results are from one model fit and evaluation seed 0.

Goal generation and planning. Reset states receive seven random warm-up actions. Six further random actions generate a reachable goal image. CEM optimizes a length-six action sequence in latent space, with population 512, 32 elites, and six iterations. The learned planner uses the goal image, without privileged position coordinates or the generating actions in its cost. It executes the entire selected sequence without observation feedback. Success is equality of final player positions. Goal generation, random actions, and the selected plan use the same future simulator random keys; replaying the generating actions therefore reaches all goals and serves only as a protocol check. This protocol samples nearby reachable goals, not arbitrary distant tasks.

The planned endpoint position L1 error is 0.0781, against 1.5313 for random actions. Pixel MSE on the 0–255 scale is 104.00 versus 1477.92. These quantities use the same 128 starts as the reported $121/128$ versus $24/128$ reach counts. They do not quantify variation across fitted models.

Reward control and head generalization. Reward and value heads use the first 80,000 transitions (320 time rows across 250 streams) from the world-model training archive. Held-out time rows are excluded from head fitting but not from encoder training. Value fitting uses the first 80 time rows owing to truncated-return handling. Reward and value fitting correlations are 0.922 and 0.995; held-out correlations are 0.353 and 0.266. Held-out reward RMSE is 0.222. Mean predicted held-out value is 2.356, versus mean empirical discounted return 1.348.

Reward evaluation uses 128 starts, 100 steps, no autoreset, and rewards accumulated while alive. The CEM controller executes one action and replans from each real observation. Random actions, myopic fitted-reward argmax, one predicted step plus value, and CEM obtain returns 1.4453, 0.7813, 0.9438, and 0.4734, respectively; their across-start standard deviations are 1.0897, 1.1109, 1.2014, and 1.1039. Fractions alive at the end are 0.6328, 0.7266, 0.6328, and 0.6719. Aggregate summaries do not preserve the per-start covariance needed for paired confidence intervals. The archive's mean reward multiplied by 100 is not an online behavior baseline and is excluded from the comparison.

Earlier Craftax experiments used a different 81×32 spatial latent and different horizons. They are not merged with these 512-dimensional results. There is no matched online flow-controller comparison for this checkpoint.

J Shared-policy comparison: protocol

This is the protocol behind Section 5 and the behavioral rows of Section 4. It was fixed before any of its policy outcomes existed (`research/shared_decoded_cnn_protocol.md` in the repository).

Matrix. Four worlds: per-frame gray noise without wind, per-frame gray noise with wind, persistent gray noise with wind, and the gate. For each world, the grid JEPA and the flow model each train three policy seeds (0, 1, 2) inside imagination: 24 policies. The gate additionally receives three decoded-real control policies per family: 6 policies. All 30 outcomes are reported. Gray-world JEPA checkpoints are model seed 0; the gate JEPA is model seed 1, preselected on prior development evidence that its decoded-real CNN could discover the strategy. Every flow checkpoint is the seed-0 fit. Policy seeds are not independent world-model fits.

Policy and optimizer. The policy is the unmodified pixel actor-critic of the original study: a three-block Impala CNN on native 63×63 RGB, a GRU that also receives the previous action, a

categorical actor and a 64-bin histogram critic; 1,702,980 parameters. Both families use the same recurrent PPO: AdamW at 2.5×10^{-4} with weight decay 0.01, four epochs, minibatch 128, 16-step truncated backpropagation, $\gamma = 0.99$, GAE $\lambda = 0.9$, value coefficient 0.5, entropy coefficient 0.01, clipping 0.2, global gradient clipping 1.0, batch 512 and horizon 48. The three non-gate worlds use 50 updates (1,228,800 positions, 2,400 optimizer steps); every gate policy, imagined or decoded-real, uses 150 updates (3,686,400 positions, 7,200 optimizer steps). The training script refuses any other budget unless run as an explicitly non-scientific two-update smoke.

Observations. Each policy sees only its own model's decoded RGB: decoded posterior encodings of real frames during burn-in and evaluation, decoded model predictions in imagination. No raw pixel reaches the actor after the encoder. Training starts are sampled uniformly from valid eight-frame archive histories with no reset in the history; burn-in processes the first seven decoded frames with their real incoming actions. Latents keep the native $4 \times 4 \times 32$ interface without resizing, whitening or amplification; decoder quantization is retained.

Task signals. Flow uses its native sampled dynamics at four function evaluations and its jointly fitted reward and termination heads. The JEPA uses its deterministic predictor, the existing 80,000-frame decoder, and reward and termination heads of the original architecture refitted on all 500,000 training transition labels with the original 6,000-update recipe. The refit never updates encoder, predictor or decoder. Equal label counts do not equalize supervision stage: the flow heads send full-strength gradients into the dynamics trunk for the first half of its training and 0.1-scaled gradients afterwards, while JEPA world-model fitting is reward-free.

Evaluation. 256 fresh reset episodes are admitted by surviving seven uniformly random actions; the admission never looks at the policy. The same accepted prefix identities and all environment and action keys are reused across families and policy seeds, so every contrast is paired. The actor then controls 121 steps under the 128-step cap. Survival is return above -0.5; walls passed counts wall exits before or on the terminal transition. Every policy is its final checkpoint. Native-RGB trajectories of the first three accepted episodes are stored for each policy regardless of outcome.

Validation before the runs. Independent CPU checks of the training and evaluation code; GPU codec parity between the imagination path and the evaluation path for every checkpoint (context versus single-step encoding, 256- and 512-frame batches, head-cache encoding); unchanged frozen-state hashes; exact gate-state reconstruction; shared initial-context and evaluation-packet fingerprints. Two-update smokes per family and path supplied wall-clock estimates used only for scheduling.

State-blind reference rows. Two rules that never read an observation are run through the same paired evaluator on the same packets: uniform random actions, and strict left–right alternation, which is the trivial survival strategy of the still worlds. They calibrate the survival numbers of Table 3. Alternation survives the per-frame still world in every episode; both rules survive no episode of either wind world or of the gate, with control lengths of 17–18 steps in the wind worlds and about 24 in the gate. A trained policy whose control length falls below these values is steering into the boundary faster than chance.

Imagined termination probe. To separate what the policy learns from what its training environment shows it, each frozen model is rolled forward for 48 steps from 256 uniformly sampled real eight-frame contexts under the recorded actions (the data policy is uniform random), and its own termination head is read on two inputs: the model's predicted latent, exactly as in imagination, and the encoder latent of the real frame at the same step. The recorded termination labels give the ground truth. Nothing is trained and the adapters are byte-identical to those of the campaign. Figure 3 plots the implied probability that an imagined episode is still alive after k steps against the recorded survival. Per-step outputs, including latent distances between predicted and encoder latents, are stored with the campaign results.

K CRAFTAX INVENTORY COMPARISON: PROTOCOL

The protocol behind Section 6; fixed before the policies were trained (`research/craftax_native_inventory_protocol.md` and `research/craftax_inventory_protocol.md`).

Environment. Craftax-Classic-Pixels version 1.5.0 with unchanged defaults: 17 actions, 64×64 procedurally generated maps, a 10,000-step native episode limit, 22 achievements. The native reward is the change in the achievement count plus 0.1 times the change in health; the evaluator checks the telescoping identity between summed rewards and terminal achievements and health on every episode. Termination occurs at the step limit, on lava, or at non-positive health. The evaluator uses the no-auto-reset environment, stops each episode at its first native termination and freezes its metrics.

World models. Both models are trained on the same 500,000-transition archive of uniformly random play under the same ten-minute budget with the same $4 \times 4 \times 32$ latent shape. The JEPA is the 512-d grid model with one-step factored predictor and eight-frame history; its pixel decoder and its reward and termination heads are refitted afterwards on the frozen latent with a fixed recipe (decoder: 8,000 steps, batch 256; heads: 6,000 steps, batch 1,024, hidden 512) whose arguments and resulting hashes are checked at load time. The flow model is the native recipe with jointly fitted heads. The frozen model weights of the JEPA are hash-verified to be byte-identical before and after head fitting.

Policy training in imagination. The same pixel CNN and GRU policy as in the cube worlds, with the original Craftax pixel-PPO settings, is trained for a nominal five million imagined steps (4,998,400 executed) inside each model from a shared bank of genuine eight-frame episode-start contexts. The JEPA adapter reads reward and termination from (z_t, a_t), predicts once per action and samples termination from the head's probability; the flow adapter runs its native expanding-window rollout with four function evaluations. Imagined episodes are capped at 1,024 steps. No behavior cloning, action masking, reward alteration, resource injection or checkpoint selection is used; a non-finite loss fails the run rather than being skipped. Three policy seeds per family, one model seed.

Evaluation. Each final policy is evaluated on 256 native episodes with evaluation seed 123. Environment keys derive from episode identity and step only, action-sampling keys from a separate stream; the construction is stored with the results. The policy observes native RGB and its previous action. Primary endpoint: the fraction of episodes in which the wooden-pickaxe achievement is true at the end of the episode. Retained: all 22 achievement rates with Wilson intervals, achievement count, native return, executed steps, survival, termination causes, first-acquisition steps, final inventory, and the terminal native score where a native termination occurred. Uncertainty intervals are across evaluation episodes for one fixed policy and do not estimate variation across model fits.

L EARLIER UNMATCHED GATE EXPERIMENTS

The experiments in this appendix predate the matched shared-policy protocol of Appendix J: real and imagined policies did not share evaluation keys, latent-MLP and decoded-CNN policies are mixed, and the summary tables (Table 14) are regenerated from the last evaluation of each run. None of 27 policies trained inside the JEPA's imagination discovered the gate strategy, including nine that used a sampling head fitted on the predictor's residual and nine that averaged eight of its samples. An exact nine-branch calculation on frozen encoders shows the scoring operator of Equation 2 changing the optimal action for 1–3% of context–goal pairs (Appendix R); it is a small direct illustration, not a control result.

Reference model. For the gate, a reconstruction-trained tokenizer and causal flow-matching predictor provide a feasibility reference (Lipman et al., 2023; Chen et al., 2024; Hafner et al., 2025). It uses the same latent shape, data, and nominal model-training budget, with a 32-frame context and jointly fitted reward and termination heads. Architecture, supervision, and history length differ from the JEPA. One dynamics checkpoint is evaluated, so its policy runs show variation conditional on that fit rather than variation across fitted flow models.

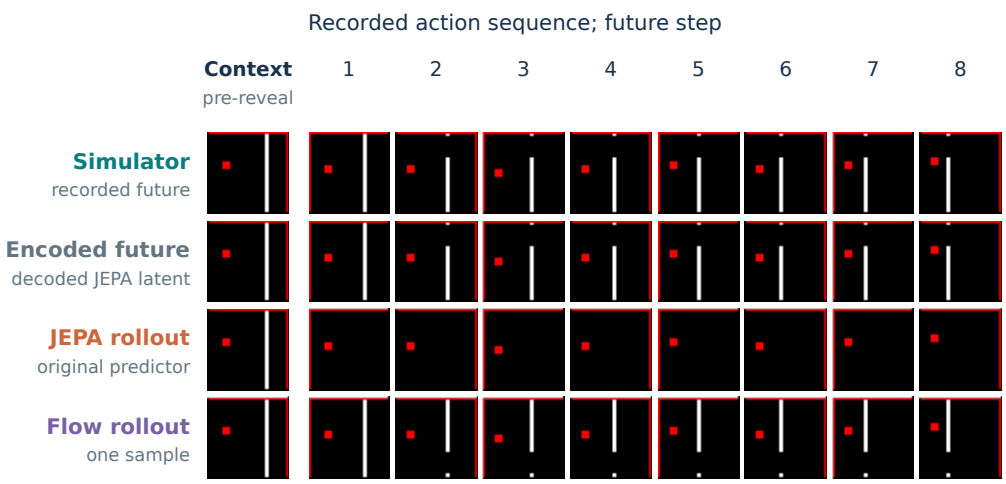


Figure 8: **A wall visible in real encodings disappears in predicted rollouts.** A pre-reveal context is followed by futures under recorded actions: the simulator, decoded real JEPA encodings, decoded original JEPA predictions, and one flow-model sample. The JEPA rows use encoder seed 0 and the same fitted decoder. The example is the first stored diagnostic context at wall position $x = 45$ among 64 contexts whose recorded futures survive the horizon. The flow sample is a single-checkpoint feasibility reference. A decoding failure on predicted latents can reflect both prediction error and off-distribution read-out; it is not alone proof of information loss.

L.1 Useful wall information remains in the frozen representation

Decoded real encodings visibly retain the approaching wall (Figure 8). To test whether that information can support behavior, we train a convolutional policy in the real gate on the frozen decoder's output. Two of nine runs discover the survival strategy, both using encoder seed 1; the corresponding real-pixel CNN discovers it in two of three runs (Figure 9). This establishes useful information in at least one fitted encoder under the decoder-and-CNN read-out. The decoder adds reconstruction supervision outside the world-model budget, so this is a test of recoverable information rather than unsupervised control sufficiency.

The policy interface matters. With an MLP torso, real pixels yield 0.91 walls per episode and frozen latents yield 0.94, with no discoveries in either group. An MLP on decoded latents similarly yields 0.92 walls. Thus failure of this MLP training recipe is not evidence that the encoder has lost the obstacle. These comparisons do not set a capacity limit for all MLP policies; they locate a limitation of the tested read-out and optimization procedure.

L.2 The tested imagination pipeline does not discover the strategy

In the earlier unmatched protocols, all three JEPA predictor variants produce poor downstream control (Table 14; per-seed outcomes in Appendix G). The original predictor yields 0.27 walls per episode; one residual sample and the mean of eight each yield 0.26. No policy among these 27 runs discovers the strategy; the highest final survival is 3.1%. Real-transition PPO on the same frozen encoders yields 0.94 walls. The difference measures the combined effect of learned transitions, fitted heads, and the changed PPO protocol. Even the stronger real-latent MLP result falls far short of reliable survival.

The flow reference supports feasibility at this task and nominal model budget: one of its nine policies discovers the strategy. The remaining eight pass 0.72–1.18 walls per episode. Since all nine use one dynamics fit with a different architecture and supervision, this reference does not establish a model-family ranking. Training the gate JEPA for twice and four times the steps also leaves imagined-policy survival at 0–3% in the tested extensions (encoder seed 0, policy seed 0, each predictor variant).

Table 14: **Final gate outcomes, with predictor variants kept separate.** Mean ± sample standard deviation across policy runs; discovery is final survival above 50%. JEPA imagination variants and real-latent conditions each contain 3 models × 3 policies; pixel controls have three policies. Real training changes rewards, terminations, and PPO protocol alongside dynamics. Flow is a single-model reference.

Training / observation	Predictor or torso	Walls	Discovery
Imagined JEPA	Original	0.27 ± 0.22	$0/9$
	Residual sample	0.26 ± 0.32	$0/9$
	Residual mean of 8	0.26 ± 0.30	$0/9$
Real / frozen JEPA	MLP	0.94 ± 0.04	$0/9$
Real / pixels	MLP	0.91 ± 0.08	$0/3$
Real / decoded JEPA	CNN	1.57 ± 1.26	$2/9$
Real / pixels	CNN	2.91 ± 1.65	$2/3$
Imagined flow	Sample	1.45 ± 1.23	$1/6$
	Mean of 8	0.88 ± 0.14	$0/3$

L.3 Prediction and fitted heads provide distinct failure signals

The same decoder that renders real encoded walls removes them from original predicted rollouts. Across all three encoder seeds, decoded shared-wall intensity is at most 0.02 by the second predicted step (Appendix G). The shared wall pixels occur in both possible gap futures, so a pixelwise mixture of those futures would retain them. This observation rules out that simple pixel-space picture; it does not determine what averaging or distribution shift does in the learned latent space.

A within-encoder counterfactual diagnostic makes the scale of the feature explicit. On 64 pre-reveal contexts for seed 0, the first-step squared distance between wall-present and wall-absent encodings is 0.035, while total teacher-forced prediction error is 0.145. The same ordering holds for all three seeds (Appendix G). This comparison concerns sensitivity to wall removal versus error in all features. It neither identifies a causal mechanism nor permits comparing information content through raw distances across independently fitted latent spaces.

The fitted heads expose another limitation. For encoder 0, termination AUC is 0.96 for border deaths but 0.65 for wall deaths. Predicted rewards for the two death causes are -0.89 and -0.05, respectively, against the true -1. An imagined policy therefore trains on a weak wall-collision signal. The residual-sampling variants retain these heads and the policy protocol. Their poor control outcomes therefore test this particular intervention, not distributional prediction in general.

M Matched-control protocol and outcomes

The primary matrix contains three fixed gate model checkpoints, five policy seeds (0–4), and three transition/signal conditions. Within each model/policy block, initialization, sampled starts, source identities, checkpoint hash, data hash, requested updates, policy parameter count, evaluation seed, and episode count must match. Start arrays are compared by semantic content hashes. Only complete three-condition blocks with 150 finished updates, finite required training diagnostics, and scalar outcomes reproduced from saved episode arrays enter the complete analysis. Missing, partial, or invalid runs retain their place in the planned denominator.

The primary endpoints are wall exits before or on the terminal transition and survival through the 128-step cap. A same-step wall exit counts even if that transition is terminal. A death on step 128 is counted as death, using episode return rather than duration. Survival above 50% is a secondary operational threshold, not independent evidence of a specific learned maneuver. Each saved final policy is evaluated on 256 real episodes with a common key schedule. Outcomes are paired within model and policy seed. Raw hidden state is used only to reconstruct the simulator transition source and must reproduce sampled current pixels exactly before training proceeds.

Within a fixed model, policy-seed effects are equally weighted. The main descriptive interval uses 10,000 paired-policy bootstrap resamples with analysis seed 7341, conditional on the evaluation set.

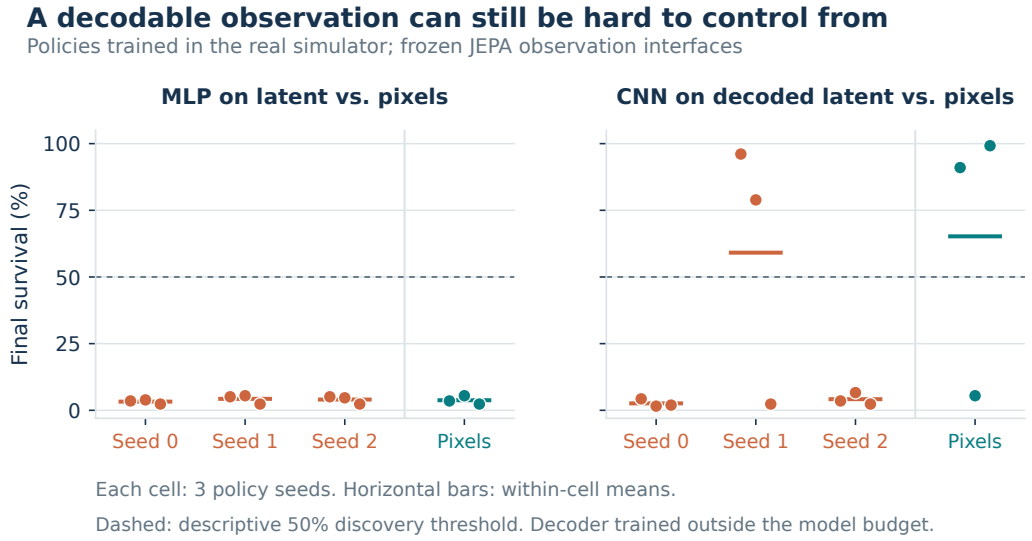


Figure 9: **Recoverable information can support successful control.** Final real-gate survival for MLP policies on frozen latents or pixels (left), and CNN policies on decoded latents or pixels (right). Points are policy runs, grouped by encoder seed; the dashed line is the 50% discovery criterion. Both decoded-latent discoveries use encoder 1. Reconstruction supervision and CNN training are additional to representation fitting.

A separate interval resamples evaluation slots jointly across policies while holding fitted policies fixed. It measures evaluation variation, not policy-training uncertainty. Across model fits, we show all three effects, their equal-model mean, and range; we do not treat 15 policy pairs as 15 independent models or report a model-family interval. All intervals are unadjusted and descriptive.

Exact-simulator states become absorbing after true termination. When fitted signals are used, their learned continuation can remain positive there; post-terminal alive mass and weighted reward are reported as part of the intervention's support change.

M.1 Completed primary outcomes

All 45 prescribed MLP runs passed full-budget, pairing, checkpoint-identity, and per-episode outcome checks. Their primary outcomes appear in Table 15. Table 16 retains every predeclared wall-count contrast with its within-model policy-seed interval. No confidence interval here estimates variation over a model family.

N Native-grid policy sensitivity

The sensitivity uses the same frozen models, rollout protocol, and three transition/signal conditions, with three policy seeds per model. The policy applies a CNN to the existing $4 \times 4 \times 32$ topology and has 583,236 parameters. All 27 runs complete 150 updates and 1,228,800 requested transitions. Nine three-arm start-array pairs, model identities, frozen source and policy hashes, and every scalar endpoint are verified from saved artifacts.

Within-model exact/true minus learned/fitted wall-count effects are 0.616, 0.083, and 0.250; the corresponding paired-policy bootstrap intervals are $[0.516, 0.691]$, $[-0.312, 0.395]$, and $[-0.148, 0.477]$. These coarse intervals use only three paired policy seeds and condition on each checkpoint and its fixed evaluation set. They are not pooled with primary MLP intervals.

Table 15: **The exact training environment does not make this PPO recipe solve the gate.** Final real-environment outcomes, mean ± sample standard deviation across five policy seeds per row. Each of three fitted models receives all three matched conditions. None of the 45 policies exceeds 50% survival. "Fitted" means the unchanged learned reward and termination heads; "true" means simulator signals.

Model	Transitions / signals	Walls per episode	Survival (%)
0	Learned / fitted	0.285 ± 0.265	0.94 ± 1.28
0	Exact / fitted	0.230 ± 0.366	0.62 ± 0.98
0	Exact / true	0.452 ± 0.099	0.47 ± 0.33
1	Learned / fitted	0.217 ± 0.356	0.78 ± 1.35
1	Exact / fitted	0.068 ± 0.152	0.16 ± 0.35
1	Exact / true	0.341 ± 0.210	0.39 ± 0.39
2	Learned / fitted	0.320 ± 0.196	0.39 ± 0.28
2	Exact / fitted	0.096 ± 0.119	0.00 ± 0.00
2	Exact / true	0.543 ± 0.119	1.41 ± 0.45

Table 16: Matched primary wall-count differences. Intervals are 95% percentile bootstraps over five paired policy seeds within one fixed model, conditional on its evaluation set. They are unadjusted descriptive intervals; all three prespecified contrasts are retained.

Model	Contrast	Δ walls	95% interval
0	Exact/fitted − learned/fitted	-0.055	$[-0.368, +0.259]$
0	Exact/true − exact/fitted	$+0.222$	$[-0.109, +0.452]$
0	Exact/true − learned/fitted	$+0.166$	$[+0.015, +0.348]$
1	Exact/fitted − learned/fitted	-0.149	$[-0.492, +0.045]$
1	Exact/true − exact/fitted	$+0.273$	$[-0.027, +0.498]$
1	Exact/true − learned/fitted	$+0.123$	$[-0.195, +0.437]$
2	Exact/fitted − learned/fitted	-0.223	$[-0.429, +0.046]$
2	Exact/true − exact/fitted	$+0.447$	$[+0.329, +0.598]$
2	Exact/true − learned/fitted	$+0.223$	$[+0.034, +0.420]$

O FORECAST-READOUT PROTOCOL AND OUTCOMES

The three gate models stay frozen. Up to 16,384 training contexts and 8,192 evaluation contexts are sampled without replacement with seeds 20260919 and 20260920, using matching context IDs across models. Past context may not cross a reset. Forecasts use the original recorded outgoing actions, with horizons 1, 2, 4, and 8. Parameters and BatchNorm statistics must remain unchanged during extraction, and cached encodings are checked against the public rollout implementation.

The three feature domains are real future encoding, autoregressive forecast, and copied current encoding. Each domain has 512 coordinates. An affine ridge read-out uses mean squared error plus $10^{-3}\|W\|^2$, with unpenalized intercept and no hyperparameter search. Both declared scaler analyses are retained: current-training-latent mean/standard deviation shared across domains; and source-domain training mean/standard deviation for each horizon. Both use a fixed 10^{-6} scale floor, and every transfer applies its source-fitted scaler. No evaluation data determine a scaler, regularizer, checkpoint, or reporting condition.

Cube coordinates are divided by 63 for fitting and reported in pixels. Wall position is fitted and evaluated only on truth-present rows; the absent-wall sentinel is never regressed as a coordinate. Presence reports class prevalence, AUROC, average precision, and threshold accuracy/balanced accuracy, with undefined one-class metrics omitted. Every task includes a constant training-label-mean baseline and sample counts. A task with fewer than 64 fitting rows is marked insufficient.

The archive stores reset frames after terminal transitions, so physical-state read-outs use a horizon-specific continuing-episode mask. In particular, one-step evaluation does not select for eight-step

Table 17: Native-grid CNN sensitivity, kept separate from the primary MLP campaign. Each cell is mean ± sample standard deviation across three policies from one fixed model. None of the 27 policies exceeds 50% survival.

Model	Transitions / signals	Walls per episode	Survival (%)
0	Learned / fitted	0.000 ± 0.000	0.00 ± 0.00
0	Exact / fitted	0.431 ± 0.436	2.08 ± 1.85
0	Exact / true	0.616 ± 0.090	1.17 ± 0.39
1	Learned / fitted	0.284 ± 0.246	0.65 ± 0.60
1	Exact / fitted	0.738 ± 0.067	2.34 ± 0.68
1	Exact / true	0.367 ± 0.238	0.52 ± 0.45
2	Learned / fitted	0.202 ± 0.158	0.00 ± 0.00
2	Exact / fitted	0.229 ± 0.370	0.39 ± 0.68
2	Exact / true	0.452 ± 0.256	0.65 ± 0.81

survival. The hidden-gap negative control instead retains every pre-reveal context, including later resets, because future-survival selection could reveal the gap through the recorded actions. The gap label refers to the original current wall. Chance forecasting of this unavailable bit is compatible with correct treatment of uncertainty.

All fitted weights, scalers, selected IDs, labels, masks, per-example predictions, and provenance hashes are saved. Overlapping windows are not independent samples; any uncertainty uses evaluation stream or episode clusters. A weak read-out remains consistent with information loss, poor conditioning, limited support, or insufficient probe capacity.

Execution and uncertainty. All three models use the same canonical padded frame batch and predictor batch. A strict parity check against the public rollout function is bit-exact at the production shape, and model parameters and BatchNorm statistics remain unchanged. This canonicalization was fixed after an initial smoke parity failure and before any successful read-out fit; no tolerance, model weight, or metric was changed. The analysis recomputes 1,080 metric rows and 720 paired contrasts from saved float32 predictions without refitting. Intervals resample the 100 evaluation environment streams jointly across compared read-outs, with 2,000 replicates and seed 20260921. They condition on the fitted model and probe. AUROC and average precision are reported as point estimates.

For the model-2, horizon-eight pre-reveal cohort, the forecast-versus-copy MAE difference is -0.63 pixels with interval $[-1.41, 0.13]$ under source-specific scaling. The transfer-versus-adapted difference is 3.85 pixels with interval $[3.48, 4.22]$. Common scaling gives -0.64 $[-1.42, 0.12]$ and 3.89 $[3.52, 4.25]$, respectively. These are descriptive conditional comparisons; the strong adaptation result occurs in one of the three fitted models.

O.1 Read-out errors and a post-hoc tracking audit

The predeclared absolute-error table is retained below for every fitted model and both scaling analyses. Its global training-mean baseline is evaluated on the selected pre-reveal slice; it is not a slice-specific optimal constant. A separate audit of saved predictions examined correlations and RMSE without refitting or modifying any prediction. The model-2 forecast-fitted probe has Pearson correlation 0.246 on the full wall-present cohort and -0.175 within the pre-reveal slice. Its full-cohort RMSE is 18.65 pixels versus 13.49 for the global training-mean constant. Thus the MAE improvement should not be described as reliable tracking or successful recovery of wall position. A subsequent train-only constant-baseline audit uses the same horizon and cohort masks, with 1,509 training and 716 evaluation rows in the pre-reveal slice. The training-cohort mean position is 29.433 pixels and the median is 30. Their evaluation MAEs are 4.769 and 4.642 pixels, respectively, improving on every highlighted model-2 feature read-out in that slice. The full-cohort training median has MAE 10.560, compared with 9.112 for the forecast-fitted probe, while the latter's RMSE remains worse than the mean constant. This is an explicitly post-hoc interpretation check, with no feature-probe refitting, clipping, latent rescaling, or alteration of the declared metric outputs.

Table 18: **Read-out fitting changes measured forecast error.** Eight-step wall-position MAE in pixels on 716 pre-reveal continuing contexts. R, F, and C denote real future, forecast, and copied-current latents; arrows indicate fitting and evaluation domains. Both train-only scaling analyses are shown. The global training-mean baseline has MAE 14.24; the post-hoc training-cohort mean and median have MAE 4.77 and 4.64. The larger refitting gain occurs in model 2; all three models are retained.

Model	Scaling	R→R	R→F	F→F	C→C
0	Common	11.66	13.79	13.99	10.77
0	Source	11.37	13.77	13.99	10.77
1	Common	13.00	14.21	13.93	11.28
1	Source	12.82	14.44	13.93	11.28
2	Common	6.62	10.05	6.16	6.79
2	Source	6.53	10.02	6.16	6.79

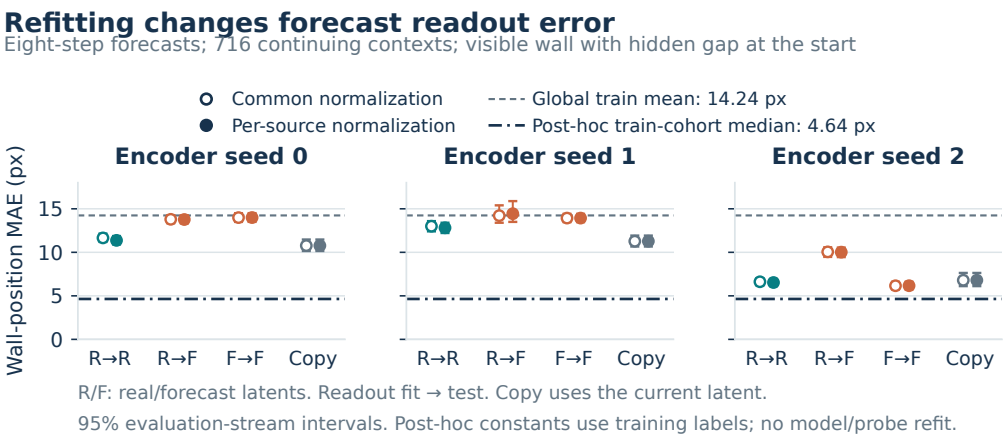


Figure 10: **Refitting changes forecast read-out error.** All three model seeds and both predeclared normalization schemes are retained. The original baseline is the global training-label mean evaluated on the selected slice; the added cohort-median constant is a post-hoc train-only check. Lower slice MAE can reflect calibration as well as tracking; the model-2 correlation audit and full-cohort RMSE prevent a reliable-position-recovery claim.

P TRIGGERED TERMINATION SENSITIVITY

This follow-up was triggered by the primary experiment's monitored post-terminal weights. It adds one condition: exact transitions, true termination, and the original learned reward evaluated on the current latent and outgoing action. It reuses every corresponding reference arm's saved start indices, checkpoint and data identity, policy initialization, update budget, and evaluation keys. The MLP stratum adds five policy seeds for each of three models; the native-grid stratum adds three, for 24 additional runs. The original 72 policies are neither retrained nor relabeled.

All added runs complete 150 updates. All 3,600 recorded training updates have finite required diagnostics and exactly zero continuation-weighted post-terminal mass and reward. Every final policy binary hash and per-episode endpoint is checked. Table 19 gives the new condition's outcomes. None of the added policies exceeds 50% survival.

The termination contrast changes continuation weighting and removes the learned after-death extension. The reward contrast holds exact transitions and true termination fixed; policies trained under different rewards can then visit different states. The latter increases the mean wall count in all six cells, while survival changes have mixed signs. Both comparisons condition on the saved

Table 19: Exact transitions, true termination, and fitted reward: the added termination-sensitivity condition. Mean ± sample standard deviation over paired policy seeds within one fixed model and torso. The two torso strata remain separate.

Torso	Model	Policies	Walls per episode	Survival (%)
MLP	0	5	0.164 ± 0.239	0.62 ± 0.90
MLP	1	5	0.216 ± 0.244	0.86 ± 1.05
MLP	2	5	0.078 ± 0.107	0.16 ± 0.21
Grid CNN	0	3	0.102 ± 0.110	0.65 ± 1.13
Grid CNN	1	3	0.000 ± 0.000	0.00 ± 0.00
Grid CNN	2	3	0.181 ± 0.313	0.65 ± 1.13

Table 20: Wall-count changes in the triggered sensitivity. "True termination" compares the new condition with exact transitions and fitted reward/termination. "True reward" compares exact transitions and true reward/termination with the new condition. Each entry gives the paired mean and its conditional 95% policy-seed bootstrap interval.

Torso	Model	True termination	True reward
MLP	0	-0.066 $[-0.486, +0.261]$	$+0.287$ $[+0.003, +0.512]$
MLP	1	$+0.148$ $[-0.083, +0.380]$	$+0.124$ $[-0.075, +0.391]$
MLP	2	-0.018 $[-0.162, +0.094]$	$+0.465$ $[+0.312, +0.619]$
Grid CNN	0	-0.329 $[-0.785, +0.000]$	$+0.514$ $[+0.422, +0.605]$
Grid CNN	1	-0.738 $[-0.793, -0.664]$	$+0.367$ $[+0.117, +0.590]$
Grid CNN	2	-0.048 $[-0.656, +0.543]$	$+0.271$ $[+0.164, +0.453]$

encoder/head checkpoint and the given policy/training procedure. The 10,000-resample intervals are unadjusted and descriptive; three model fits do not establish a model-family effect.

Q VISUAL-GOAL CONTROL IN THE DETERMINISTIC GRAY-NOISE WORLD

All three original `gray_frame_still` grid models use the same 128 freshly generated cases and unchanged planning settings. Evaluation seed 20260920 produces native resets followed by seven random actions; a history is admitted only if it survives this past prefix. No candidate is rejected according to a future model result. The goal is sampled uniformly from safe positions on the same three-pixel action lattice, with shortest Manhattan action distance between six and 12. Mean initial L1 distance is 26.883 pixels, with range 18–36. The goal image has independently drawn gray noise and remains fixed within its case.

The planner sees genuine eight-frame histories and the RGB goal. Horizon-three MPC enumerates all 64 native action sequences and minimizes summed squared latent-goal distance over the three predictions. It executes the first action, observes the true successor, and replans, for at most 24 real steps. Horizon-one MPC is a predeclared comparison. No decoder, position probe, reward head, altered reward, latent rescaling, or privileged state enters either learned planner. History and goal encoding use a common fixed $[16, 8, 63, 63, 3]$ batch shape. Outgoing action alignment is $[a_0, \ldots, a_6, a_{\text{candidate}}]$.

The primary metric is a first exact goal hit before native death. Execution stops on the first hit or death, and all 128 cases remain in the denominator. Secondary metrics are minimum/final L1 distance, entry within three pixels, deaths, and first-hit time. Random and blind-cycle references share execution noise with each model. The privileged exact-transition reference searches the same horizon with physical squared-distance costs and rejects plans with intermediate death. It reaches every goal in the shortest number of actions; this is a protocol check, not a learned competitor.

Horizon-three exact-hit counts are 5, 5, and 9 for models 0, 1, and 2. Their minimum-distance means are 22.242, 22.945, and 22.031 pixels, against 19.406 for random actions. Native-death counts are 9, 8, and 24, against 20 for random. Horizon-one hits no goal for any model, with minimum-distance

means 23.906, 24.797, and 24.047 pixels and death counts 4, 6, and 5. The fixed cycle survives but reaches no goal. These outcomes establish failure of this short-horizon latent-cost recipe on the specified distant-goal distribution; they are not a positive planning control.

Independent reconstruction checks all 1,024 prefix frames per model, every action and native transition, reachability, exact goal draws, first-hit/death absorption, summaries, visual episode selection, and recorded hashes. Across the three fits, all cohort arrays and reference trajectories are bit-identical. The three model evaluations do not create 384 independent cases. Visual episodes 0, 1, and 2 were fixed before results, and decoded selected forecasts are marked invalid after terminal padding. The empty arena permits reactive goal movement; the task does not demonstrate a need for deep planning.

Table 21: **The original distant-goal test remains a weak result.** Initial goals are six to 12 native actions away. The same 128 goals and histories are used for all three models and references. Distances are mean L1 pixels over all cases; lower is better. Random, blind-cycle, and privileged reference trajectories are shared across model evaluations.

Controller	Exact hits	Minimum distance	Final distance
JEPA H3, model 0	5/128	22.24	27.09
JEPA H3, model 1	5/128	22.95	27.75
JEPA H3, model 2	9/128	22.03	28.55
Random	3/128	19.41	30.73
Blind cycle	0/128	24.59	26.88
Privileged reference	128/128	0.00	0.00

Q.1 Separately declared distance-response follow-up

The distance-response experiment was declared after the failed 6–12-action primary result. It characterizes the same controller's operating range and does not replace that primary. All six exact goal-distance shells—one, two, three, six, nine, and 12 native actions—use the same 128 original valid histories and all three existing model seeds. The H3 and H1 planners, 24-action budget, random/blind-cycle/privileged references, native dynamics, and encoder/predictor execution shapes remain fixed. No planner or representation parameter changes.

A newly fixed goal seed, 20260921, samples uniformly among safe reachable positions on each exact Manhattan shell. The RGB goal background is shared across the six goals for a prefix, so only object position changes. Execution noise and random actions are paired by original candidate ID. The outcome is exact first hit before death. Every model–distance cell retains its own 128-case count and Wilson interval; the 768 correlated rows are never pooled into an inferential interval. The within-three-pixel secondary is already satisfied initially on the one-action shell and is not evidence of movement there.

Case 0 at every shell is the predeclared visualization. Its full recorded native trajectory is re-rendered with the recorded actions and execution keys; padding freezes after first hit or death. The initial H12 CEM alternative was assessed but not launched. Thus any distance trend concerns the unchanged H3 controller over different requested goals, not a tuned-horizon comparison.

All three models and the final independent audit completed. The H3 hit counts at distances $(1, 2, 3, 6, 9, 12)$ are $(128, 128, 121, 25, 3, 0)$, $(128, 128, 123, 32, 2, 0)$, and $(128, 128, 124, 34, 1, 0)$ for seeds 0, 1, and 2, respectively. Table 22 keeps the H1 comparison and paired references alongside these outcomes. The independent audit checks 8,179 conditions and reconstructs 176,334 executed native transitions from raw artifacts.

R Exact variance omission in two frozen encoders

This calculation uses the existing seed-0 grid JEPA in black-background wind as its declared primary condition and persistent-gray wind as its declared secondary condition. Their independently measured real-encoding position L1 errors are 0.126 and 0.717 pixels, respectively. Those diagnostics

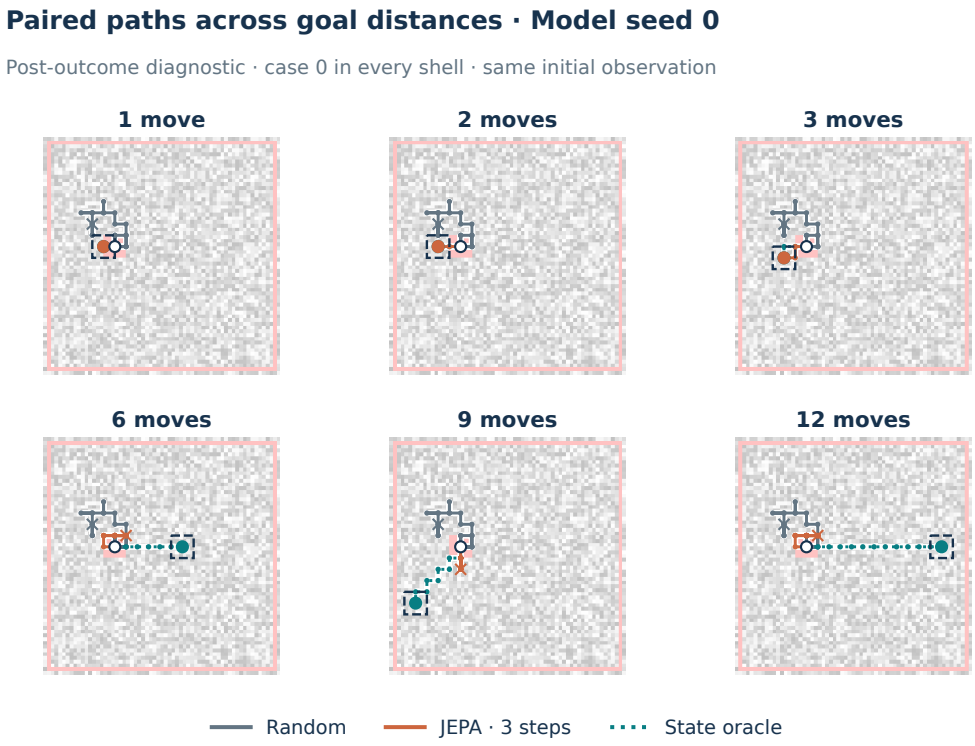


Figure 11: **Actual paths across the declared goal-distance shells.** Model seed 0, fixed case 0 in every shell. Curves are executed simulator positions, annotated only for evaluation; the planner receives RGB history and an RGB goal. Paths stop at the first exact hit, native death, or 24 actions. The examples and all six shells were fixed before the distance-response outcomes.

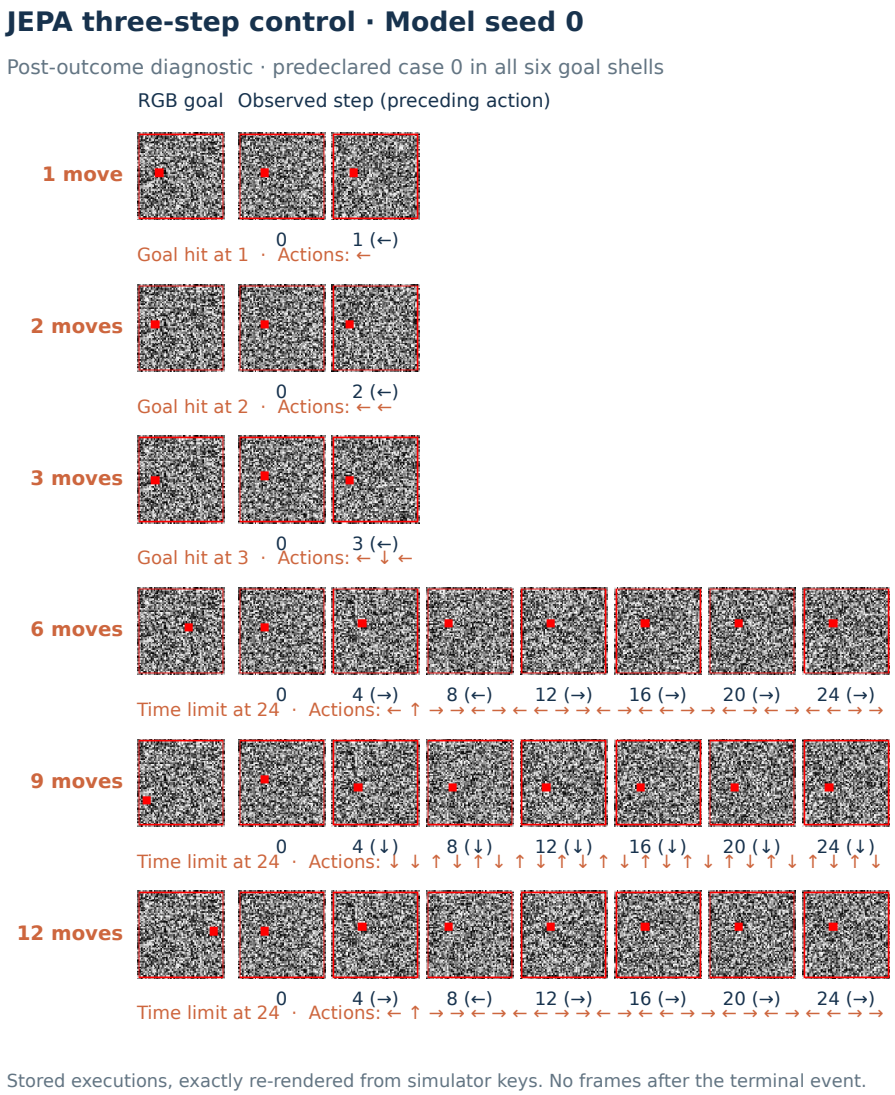


Figure 12: Native-color RGB views of the same preselected goal-distance executions for model 0. These are exact keyed re-renderings of recorded real trajectories, with no model forecast or invented continuation. The full artifact includes the corresponding displays for models 1 and 2.

Table 22: Distance-response exact-hit counts. Initial distance is measured in native three-pixel cardinal moves. Every cell has denominator 128. The same six shells, prefixes and controller settings are retained for all models; no pooled binomial interval is used.

Distance	JEPA H3, model			JEPA H1, model		
	0	1	2	0	1	2
1	128	128	128	128	128	128
2	128	128	128	39	36	46
3	121	123	124	22	16	21
6	25	32	34	4	4	1
9	3	2	1	0	0	0
12	0	0	0	0	0	0

Distance	Random	Blind cycle	Privileged reference
1	65	67	128
2	46	0	128
3	19	0	128
6	5	0	128
9	1	0	128
12	0	0	128

motivate an encoder with accessible state; no probe, learned transition model, task head, or policy is used in the variance calculation.

Fixed transition law and goals. A fresh collection uses 128 streams and 512 transitions per stream, with environment seed 20260923. From 46,136 past-valid eight-frame contexts, 1,024 are sampled without replacement using seed 20260924. Eligibility for the final calculation requires every action/innovation branch to remain inside the safe region and before the time cap. This admits the same 843 contexts in both worlds; 181 are excluded for border risk and none for the cap. Every admitted context is crossed with all 25 goals in $\{8, 18, 28, 38, 48\}^2$, giving 21,075 pairs per checkpoint. Goal images use the context's episode texture. No goal or context is selected according to model costs.

Each coordinate of the wind increment takes $-1, 0, +1$ with probabilities $0.1, 0.8, 0.1$. All nine joint labels retain their original product probabilities, including duplicate clipped wind outcomes. The simulator state and texture are used to enumerate the exact next images. Thus the resulting mean is conditioned on the full physical state and texture, not assumed equal to the mean available from a learned latent history. Current wind is independently verified from the preceding two positions and action.

Cost decomposition and action comparison. For a fixed encoder f, goal z_g, and action a, compute the nine encoded successors $z'_{a\delta}$, their exact weighted mean μ_a, and

$$M_a = \frac{\|\mu_a - z_g\|^2}{D}, \qquad J_a = \frac{\sum_\delta q_\delta \|z'_{a\delta} - z_g\|^2}{D} = M_a + \frac{\mathrm{trCov}(Z' \mid s, a)}{D}, \qquad D = 512. \tag{11}$$

All arithmetic after encoding uses float64. The independent direct-versus-decomposed calculation has maximum residual 1.33×10^{-15} and 1.67×10^{-16} in the two checkpoints. Tolerance-optimal action sets use the fixed threshold $10^{-10} \max(1, \max_a |M_a|, \max_a |J_a|)$. The primary disagreement requires disjoint optimal sets. Regret is the minimum exact-cost excess among mean-cost minimizers, so arbitrary tie breaking cannot produce the finding. All pairs, including zero-regret pairs, enter the averages.

Regret intervals are $[5.77, 9.63] \times 10^{-5}$ and $[2.02, 6.86] \times 10^{-6}$, respectively. Mean-minus-exact-latent selected actions change expected physical squared-goal cost by -0.169 pixels squared with interval $[-1.107, 0.666]$ and $+0.200$ with interval $[-0.433, 0.961]$. Both include zero; no physical-control benefit is established. The discrepancy is a modest but direct consequence of omitting variance under the specified latent metric. For this one-step quadratic objective, a mean plus the scalar covariance trace is sufficient; a full generative model is not logically necessary.

Table 23: Exact mean-only versus expected squared latent-goal scoring. Intervals resample complete collection streams, conditional on the frozen checkpoint. Latent regret uses coordinate-mean squared units and is not comparable as a physical error across independently fitted encoders.

Condition	Action disagreement (%)	Mean latent regret
Black background + wind	$2.65\ [2.19, 3.15]$	7.50×10^{-5}
Persistent gray + wind	$1.08\ [0.83, 1.36]$	3.80×10^{-6}

Controls and uncertainty. Identity-position, fixed affine-position, and deterministic-branch controls all have zero robust action disagreements. For physical position, conditional wind covariance is action independent, so its variance term cannot change the one-step squared-goal ranking. The learned nonlinear encoder can make latent variance action dependent. Canonical padded inference uses 2,048 frames, with bit-exact context and permutation parity and unchanged parameters/BatchNorm statistics. The simulator checks all sampled transitions lie in enumerated support and that all safe one-step rewards are zero.

Goals are averaged within context, then 2,000 bootstrap samples resample full collection streams; 127 streams contain admitted contexts. The intervals quantify context sampling for each fixed encoder, not training-seed uncertainty. All-branch safety selects an interior cohort and excludes terminal decisions. This experiment isolates mean-only latent-goal scoring, not survival, representation sufficiency for arbitrary tasks, learned-predictor accuracy, or recursive mean error.